



第四章：机器学习

机器学习：从数据中学习知识

机器学习的本质——找到一个函数

- 语音识别

$$f(\text{[audio waveform]}) = \text{"How are you"}$$

- 图像分类

$$f(\text{[cat image]}) = \text{"Cat"}$$

- 下棋

$$f(\text{[go board state]}) = \text{"5-5" (next move)}$$

机器学习：从数据中学习知识

机器学习的本质——找到一个函数

Step 1: 假设
一个函数，
其中带有未
知参数



Step 2: 根据
数据定义衡
量函数好坏
程度的损失



Step 3: 求解
函数中的未
知参数

机器学习：从数据中学习知识



人工智能（目标） 1950's

机器学习（技术）：从数据中自动学习一个函数 1980's

深度学习（更厉害的技术） 2010's

生成式人工智能（目标之一）：让计算机产生
复杂、有结构的内容——“创造力” 2020's

机器学习：从数据中学习知识

人工智能（目标） 1950's

机器学习（技术）：从数据中自动学习一个函数 1980's

深度学习（更厉害的技术） 2010's

生成式人工智能（目标之一）：让计算机产生复杂、有结构的内容——“创造力” 2020's

目录

- 机器学习基本概念
- 有监督学习：决策树和随机森林、kNN
- 无监督学习：聚类

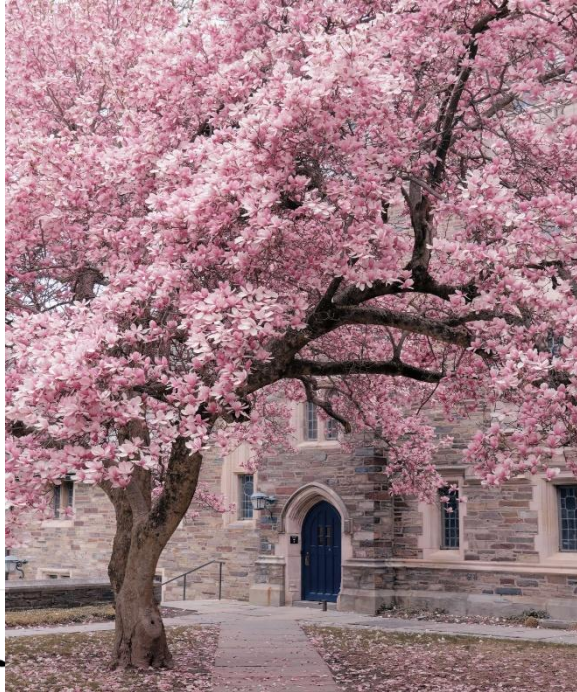
目录

- 机器学习基本概念
- 有监督学习：决策树和随机森林、kNN
- 无监督学习：聚类

这是樱花吗？



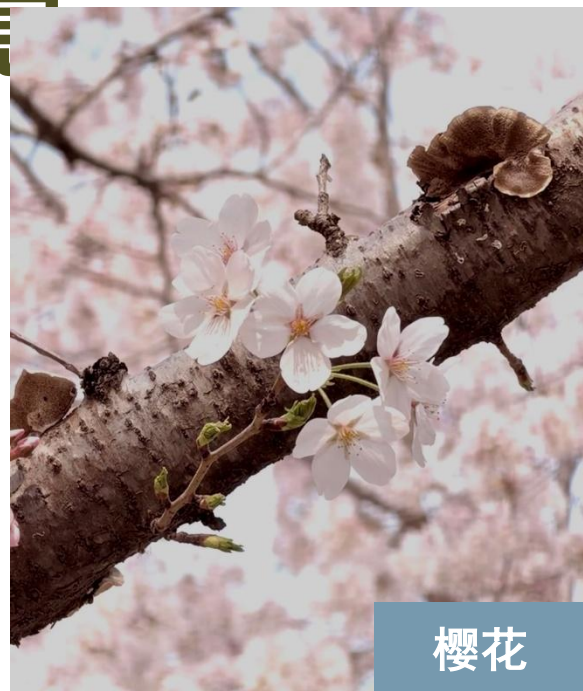
收集学习资料



收集关于它们的信息



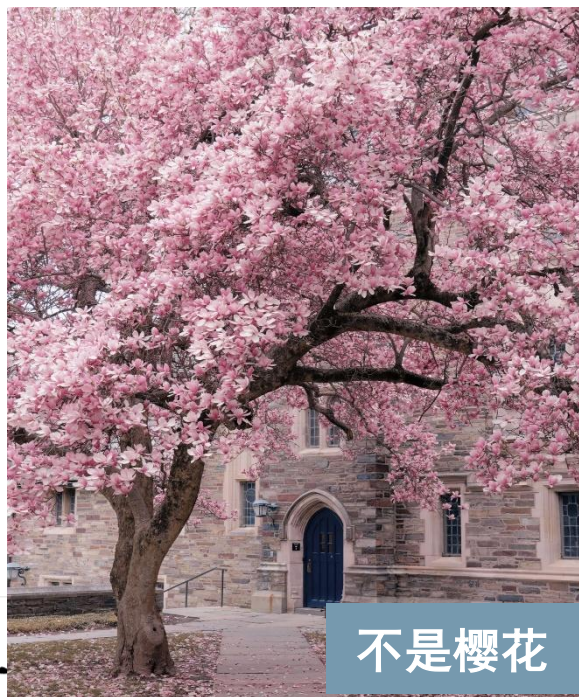
樱花



樱花



不是樱花



不是樱花



樱花



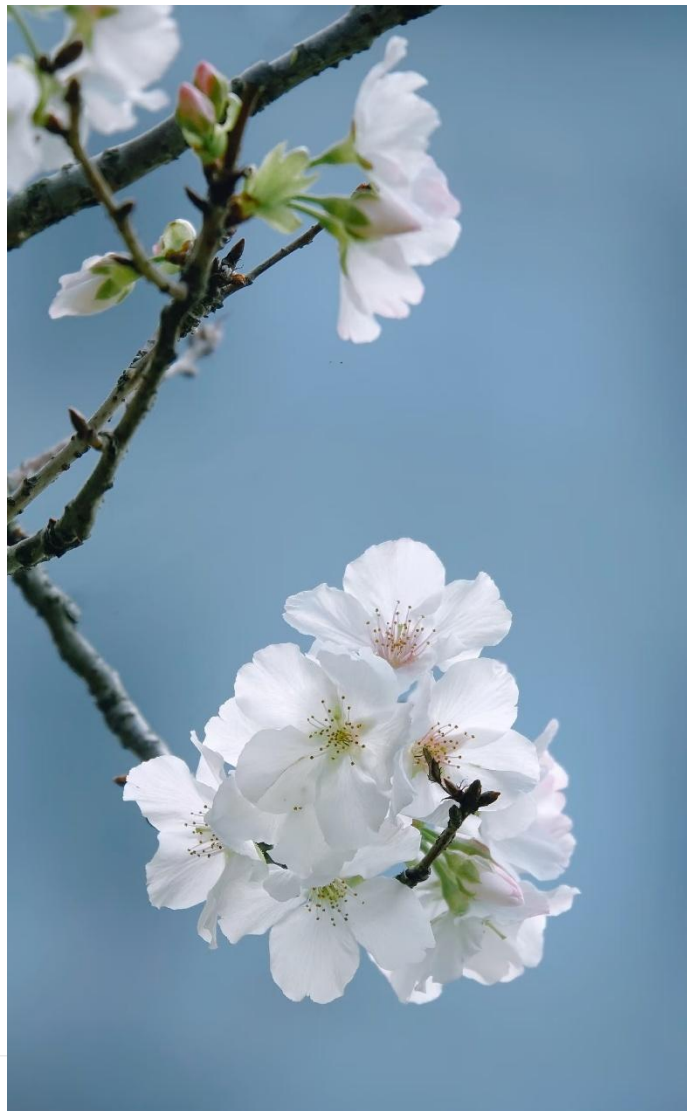
不是樱花



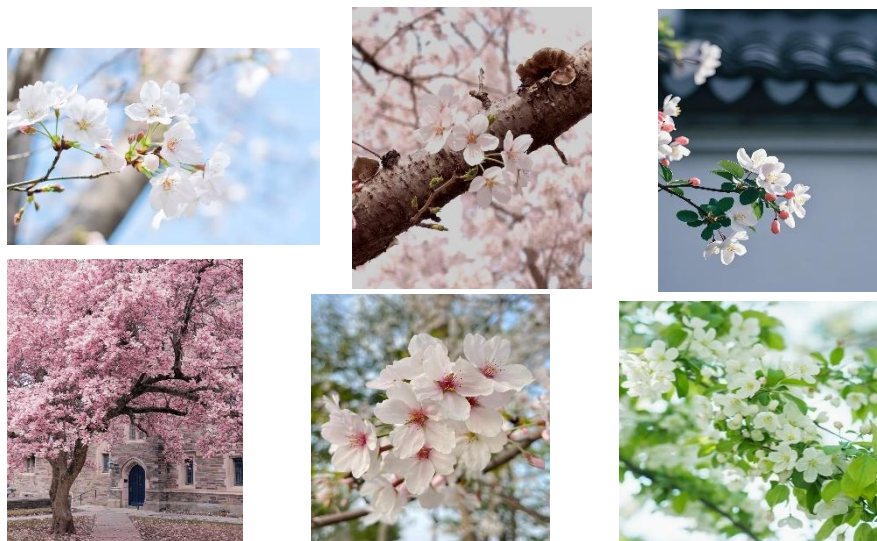
樱花：

- 有长花梗
- 花瓣有缺口
- 有的只有花 有的花叶同开
- 树干有“小嘴”
- ...

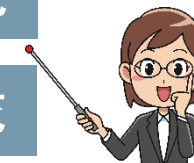
真的学会了吗



我们是怎么学会分类的



樱花	樱花	不是樱花
不是樱花	樱花	不是樱花



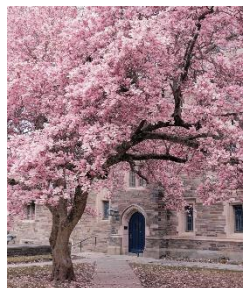
樱花：

- 有长花梗
- 花瓣有缺口
- 有的只有花 有的花叶同开
- 树干有“小嘴”
- ...



我们是怎么学会分类的

训练集



样本

樱花

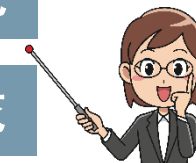
樱花

不是樱花

不是樱花

樱花

不是樱花



标记



学习器

樱花：

- 有长花梗
- 花瓣有缺口
- 有的只有花 有的花叶同开
- 树干有“小嘴”
- ...

分类器

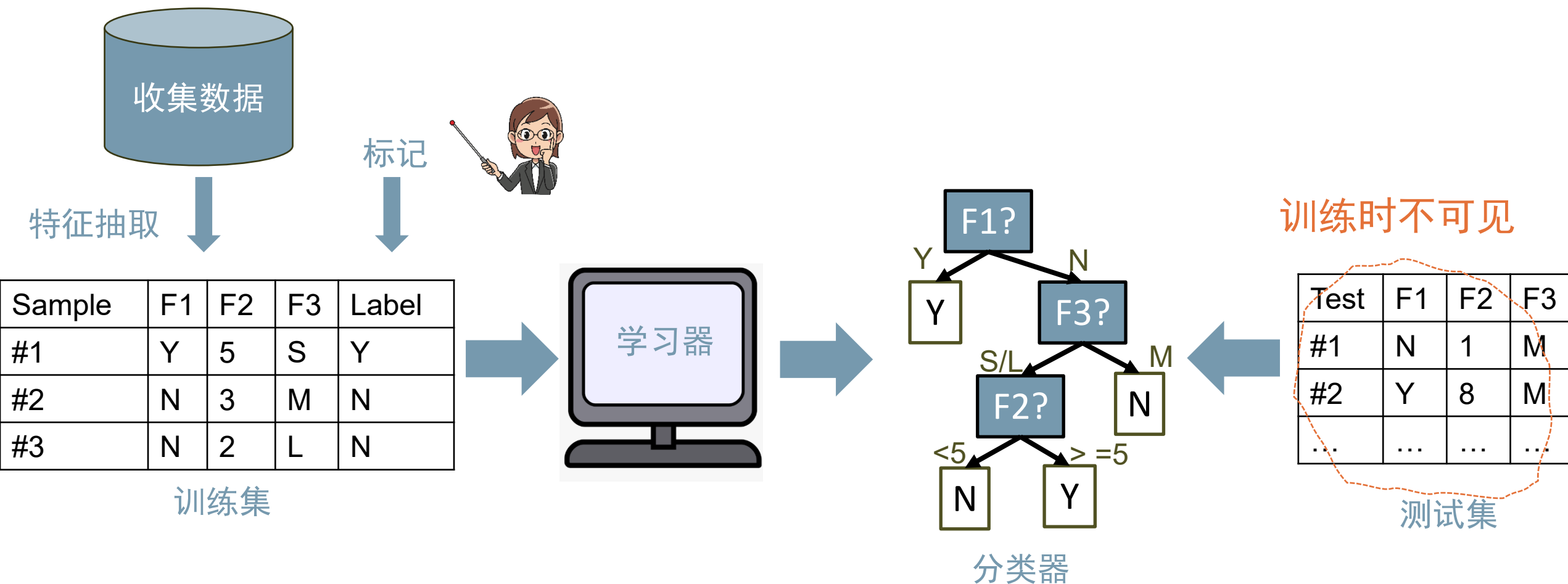
测试集



Ground truth

- **训练集**：一组样本的集合，每个样本用一组**特征**描述，**标记**表示它的类别
- **学习器**：一种可以输入训练集的算法
 - ✓ 在训练集上学习（也称为“**训练**”），输出一个分类器，即从特征到标记的映射函数
- **测试集**：没有标记的一组样本的集合
- **分类**：从训练集中学习一个分类器，使其能够将特征映射为标记，**目标**是对之前未见过的测试集，用所学得的分类器对其进行标注时，尽可能准确

分类



仅凭训练数据，如何评估分类器呢？

- 用学习到的分类器对训练数据进行分类，看看这个分类器的准确性如何？

过拟合

- 任务：判断给定的物体是否为树叶

训练集



测试集



分类器 1：不是树叶，因为它没有像训练样本那样具有锯齿状的边缘

过拟合：指分类器学习了训练数据的某些**特定**特征后，在训练样本上表现良好，但在未见过的新样本上表现不佳的现象

欠拟合

- 任务：判断给定的物体是否为树叶

训练集



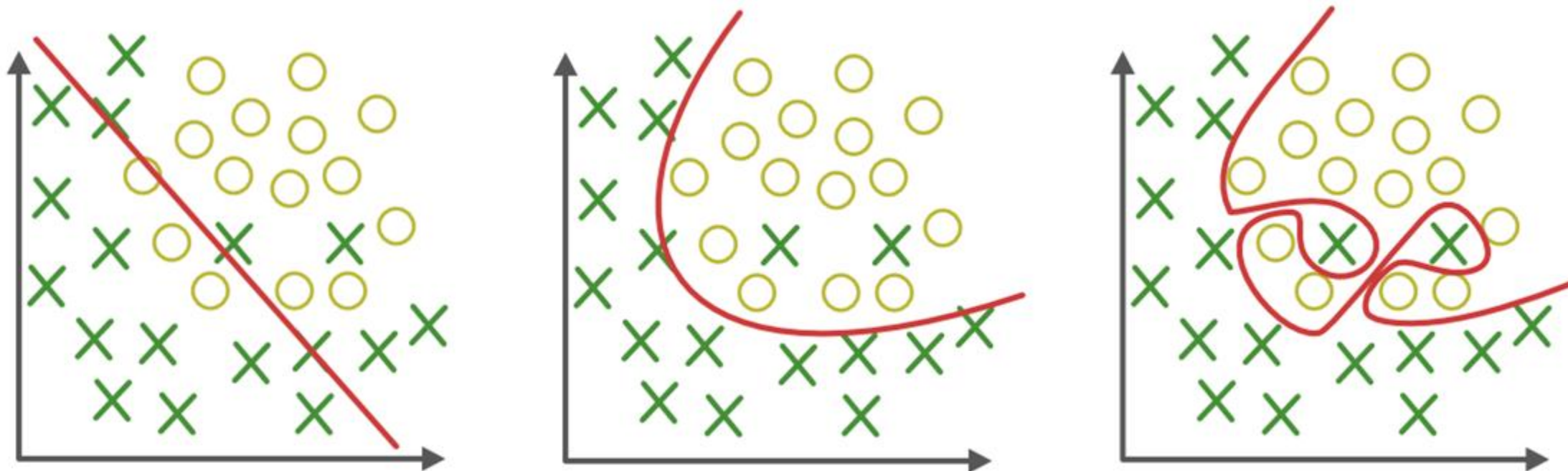
测试集



分类器 2：这是树叶，因为它呈叶子的形状

欠拟合：指分类器无法充分捕捉特征与标记之间的关系，从而导致对训练样本和未见过的新样本分类效果都不佳的现象

泛化



泛化：机器学习模型处理未见过的数据的适应与预测能力，即从训练数据中学习规律并应用到新场景的能力

经验风险与期望风险

Size in feet ² (x)	Price (\$) in 1000's (y)	$y = b + wx$ $b^0 = 0, w^0 = 0.25$	$L = \frac{1}{N} \sum_{n=1}^N e_n = 291.5$
2104	460	$\hat{y} = 526$	$e_1 = y - \hat{y} = 66$
1416	232	$\hat{y} = 354$	$e_2 = y - \hat{y} = 122$
1534	315	$\hat{y} = 383.5$	$e_3 = y - \hat{y} = 68.5$

- 训练数据集 $\{(x_i, y_i), i = 1, \dots, n\}$
- 测试数据集 $\{(x_i', y_i'), i = 1, \dots, m\}$
- 经验风险：训练集中数据产生的损失。经验风险越小说明学习模型对训练数据拟合程度越好

$$\min_{f \in \Phi} \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f(x_i))$$

- 期望风险：当测试集中存在**无穷多**数据时产生的损失。期望风险越小，学习所得模型越好

$$\min_{f \in \Phi} \int_{x \times y} \text{Loss}(y, f(x)) P(x, y) dx dy$$

经验风险与期望风险

- 期望风险是模型关于联合分布期望损失，经验风险是模型关于训练样本集平均损失
- 根据大数定律，当样本容量趋于无穷时，经验风险趋于期望风险，所以在实践中很自然用经验风险来估计期望风险
- 由于现实中训练样本数目有限，用经验风险估计期望风险并不理想，**要对经验风险进行一定的约束**
- 结构风险最小化：为了防止过拟合，在经验风险上加上表示模型复杂度的**正则化项**

$$\min_{f \in \Phi} \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f(x_i)) + \lambda J(f)$$

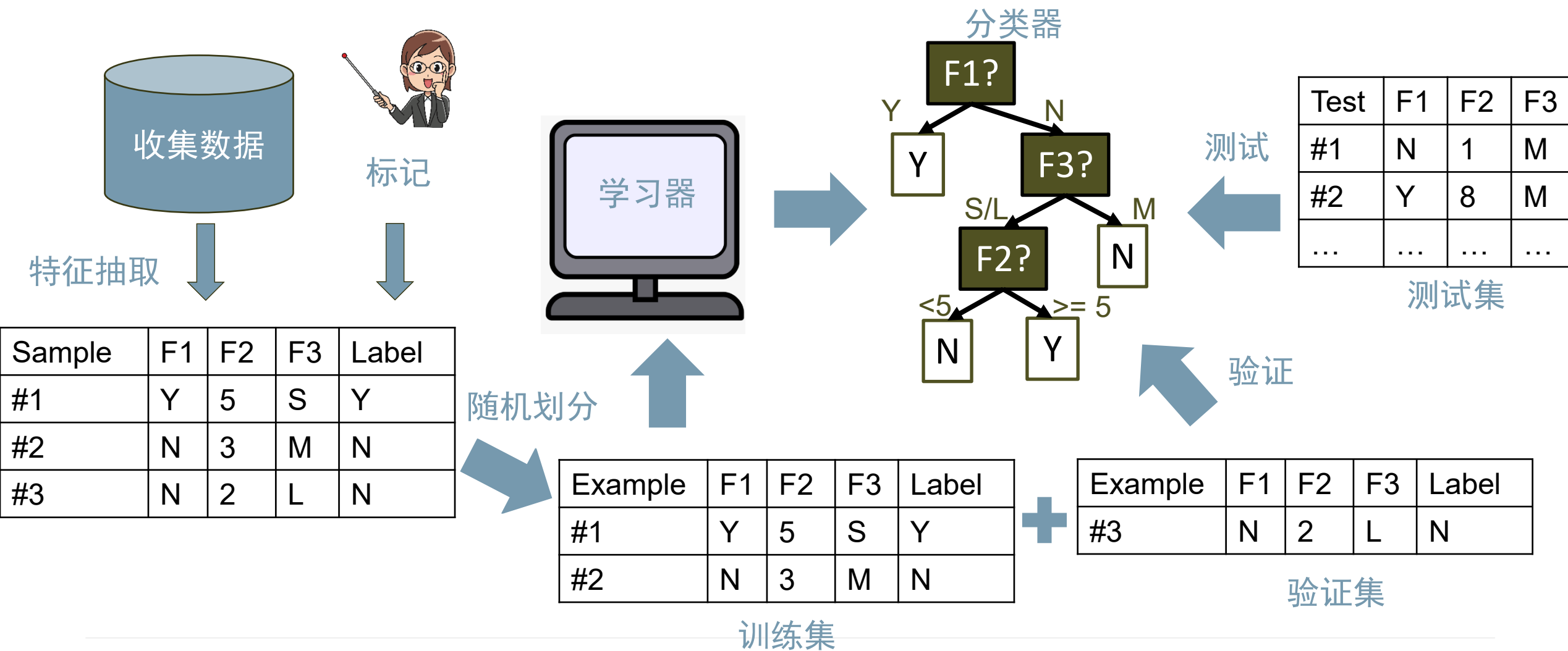
模型复杂度

在最小化经验风险与降低
模型复杂度之间寻找平衡

仅凭训练数据，如何评估分类器呢？

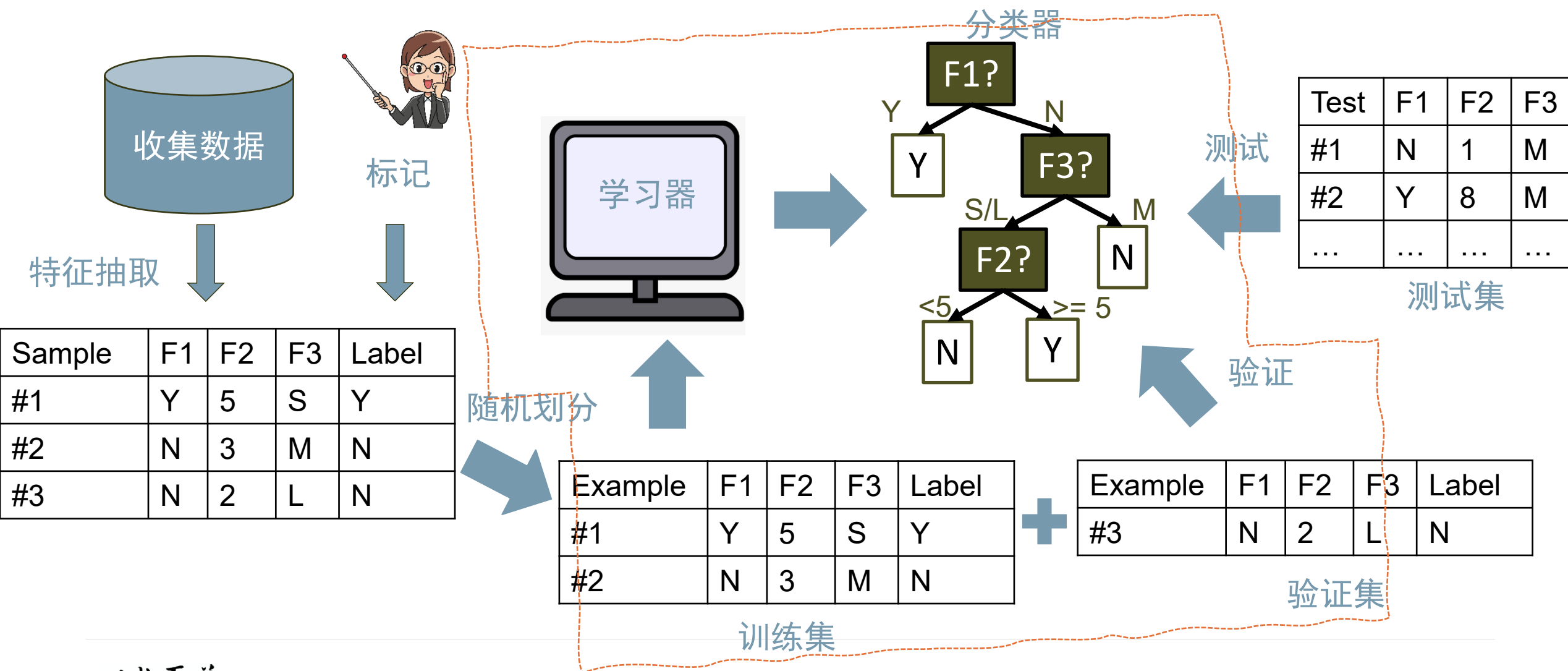
- ~~用学习到的分类器对训练数据进行分类，看看这个分类器的准确性如何？~~
- 正确答案：进行训练/验证集划分，并使用验证集来评估模型

分类



- 如果我们只进行一次训练/验证划分，那么验证结果可能会因随机性而不可靠
- 交叉验证：
 - ✓ 对于训练数据集 D ，我们将其随机划分为 K 个子集，每个子集中的实例数量大致相等： $D_1, \dots, D_K$
 - ✓ 每次 D_i 被用作验证数据，而其余的所有数据($D_1, \dots, D_{i-1}, D_{i+1}, \dots, D_K$) 用作训练数据
 - ✓ 每个验证集都会通过生成分数来评估相应的分类器
 - ✓ 将所有 K 个分数求平均值
- K 折交叉验证 (K 通常设为5/10)

分类



用交叉验证调超参数

- 超参数：用于控制学习过程的参数，其值在学习之前已预先设定并非从数据中学习而来的
 - ✓ 交叉验证通常用于调整超参数
- 确定超参数后，我们使用已确定的超参数从所有训练数据中学习分类器，然后进行测试

用于计算评估分数的指标是什么？

准确率/0-1error

- 准确率：正确预测的数量除以所有测试样例的数量
✓ E.g.: accuracy = 0.4
- 0-1 error: 1-准确率
✓ E.g.: zero-one error = 0.6

Action?	Platform	Sales (million)
Yes	Switch	1.25
No	Xbox	1
No	Switch	0.7
Yes	Xbox	1.2
No	PS4	0.95
No	Xbox	0.6
Yes	PS4	2.2
No	Switch	0.85
No	Xbox	0.75
No	Switch	0.9

正确标记

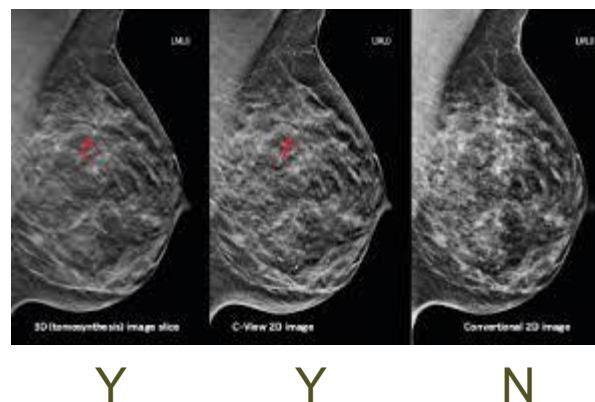
Buy?
No
No
No
No
Yes
No
No
Yes
No
Yes

预测标记

Buy?
Yes
Yes
Yes
Yes
Yes
No
No
No
No
No

准确率的局限性

- 分类任务：基于医学图像的乳腺癌检测



- 在检测出的 10,000 人中，仅有 5 人被诊断为患有乳腺癌
- 如果一个分类器将所有人一律视为“非癌症患者”，那么其准确率是多少？

准确率的局限性



- 数据不平衡：某一或某些类别在数据中的占比远低于其他类别
- 如果数据存在不平衡现象，准确率可能并非最合适的衡量标准
- 因此，我们采用F1分数

混淆矩阵

- 对于只有两类的分类任务，我们通常将其中一类称为“正类”，另一类称为“负类”
 - ✓ E.g. Yes (正类), No (负类)
- 混淆矩阵用于衡量各类别中的样本数量

	Predicted positive	Predicted negative
Ground truth positive	TP (true positive)	5 (false negative)
Ground truth negative	FP (false positive)	9,995 (true negative)

精准率和召回率

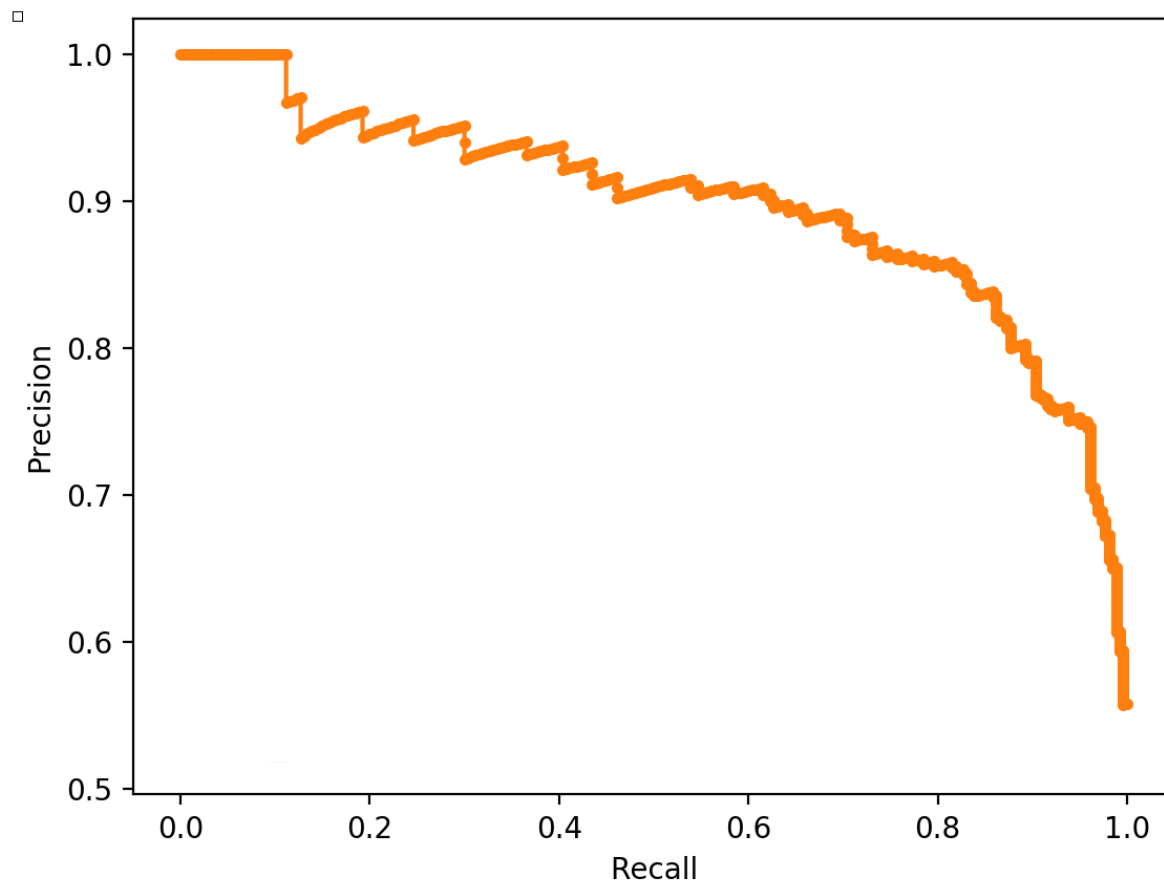
- 精准率：在所有被预测为正类的样本中，实际是正类的样本的比率
-> 尽量都找出来（查全）

$$\text{Precision} = \frac{TP}{TP + FP}$$

- 召回率：在所有正类样本中，预测为正类的比率 -> 尽量不弄错
（查准）

$$\text{Recall} = \frac{TP}{TP + FN}$$

PR曲线



如果a的曲线被b完全包住，b的性能更好

F1分数

- F1分数：精准率和召回率的调和平均值，与算术平均值相比，调和平均值是一个悲观的平均值，更重视较小值

$$F1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2TP}{n + TP - TN}$$

- 越高越好，最大为1

计算

• 混淆矩阵

	Predicted positive	Predicted negative
Ground truth positive	1	2
Ground truth negative	4	3

• 精准率 $1/(1+4) = 0.2$

• 召回率 $1/(1+2) = 0.3$

• F1

$$\frac{2}{\frac{1}{0.2} + \frac{1}{0.3}} = 0.25$$

止於至善

Action?	Platform	Sales (million)
Yes	Switch	1.25
No	Xbox	1
No	Switch	0.7
Yes	Xbox	1.2
No	PS4	0.95
No	Xbox	0.6
Yes	PS4	2.2
No	Switch	0.85
No	Xbox	0.75
No	Switch	0.9

正确标记

Buy?
No
No
No
No
No
Yes
No
No
Yes
No
Yes

预测标记

Buy?
Yes
Yes
Yes
Yes
Yes
No
No
No
No
No
No

多类分类中的准确率

- accuracy = 0.7

Action?	Platform	Sales (million)
Yes	Switch	1.25
No	Xbox	1
No	Switch	0.7
Yes	Xbox	1.2
No	PS4	0.95
No	Xbox	0.6
Yes	PS4	2.2
No	Switch	0.85
No	Xbox	0.75
No	Switch	0.9

正确标记

Interested ?
No
Moderate
No
No
Yes
No
No
Moderate
No
Yes

预测标记

Interested ?
Moderate
No ✓
No
Moderate ✓
Yes ✓
No ✓
No ✓
Moderate ✓
No ✓
Yes

多类分类中的F1分数

- 先计算每个类别的 F1 值
- Macro-F1: 求所有类别的F1平均值

正确标记

Interested ?
No
Moderate
No
No
Yes
No
No
Moderate
No
Yes

预测标记

Interested ?
Moderate
No
No
Moderate
Yes
No
No
Moderate
No
Yes

特征

- 特征用于描述对象



Colour	Taste
Green	Sour

- 利用领域知识从原始数据中提取特征的过程可能会对后续性能产生影响



特征

Item	Eat/Drink	Price (\$)	Weights (g)	State	Fresh	...
Bread	1	6	200	1	3	...
Milk	1	5	1200	0	5	...
Diapers	0	10	3000	1	1	...
Beer	1	3	300	0	3	...
Cola	1	2	300	0	1	...
Eggs	1	4	600	1	5	...
...	...	...	...	...	...	...

名词

数字

名词

顺序

TID	Mode Age (year)	Price (AUD)	Sales (million)
1	24	25	1.25
2	32	90	1
3	15	0	10
4	22	30	0.8
5	48	100	0.5

1. Max-min normalization

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Age: max 48, min 15

1. $(24-15)/(48-15) = 0.27$
2. $(32-15)/(48-15) = 0.51$
3. $(15-15)/(48-15) = 0$
4. $(22-15)/(48-15) = 0.21$
5. $(48-15)/(48-15) = 1$

Price: max 100, min 0

1. $(25-0)/(100-0) = 0.25$
2. $(90-0)/(100-0) = 0.9$
3. $(0-0)/(100-0) = 0$
4. $(30-0)/(100-0) = 0.3$
5. $(100-0)/(100-0) = 1$

特征预处理

TID	Mode Age (year)	Price (AUD)	Sales (million)
1	24	25	1.25
2	32	90	1
3	15	0	10
4	22	30	0.8
5	48	100	0.5

TID	Mode Age (year)	Price (AUD)	Sales (million)
1	-0.33	-0.55	-0.36
2	0.30	0.94	-0.42
3	-1.05	-1.12	1.78
4	-0.49	-0.45	-0.47
5	1.57	1.17	-0.54

止於至善

2. Standardization

$$x' = \frac{x - \mu}{\sigma}$$

- μ : 平均值 28.2

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- σ : 方差 12.6

$$\sigma = \left[\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2 \right]^{1/2}$$

特征预处理

- Max-min normalization
 - ✓ 当数据不符合正态分布时，此方法有用
 - ✓ 对不假定数据分布的算法来说可能也很有用
 - ✓ 范围限定在 $[0, 1]$ 之间
- Standardization
 - ✓ 当数据符合正态分布时，此方法有用
 - ✓ 不受 $[0, 1]$ 范围的限制

哪种选择更好则取决于你的具体问题以及你所使用的算法——
通过使用交叉验证来确定即可

- 维度：特征的数量
- 机器学习和数据挖掘中的维度灾难：当维度增加时，我们为了实现准确的泛化而需要的训练数据量会呈**指数级**增长

机器学习的类型

- 根据数据是否有标记，可以大致分为有监督学习和无监督学习
- 有监督学习中代表任务为回归和分类
- 无监督学习中的代表任务为聚类
- 弱监督学习：只有不完全、不确切、不精确的信息
- 自监督学习：利用海量无标注数据，通过自动构造辅助任务生成伪标记，让模型通过自身数据学到有用的语义表示
- 强化学习：在环境中通过试错，采取动作以最大化长期累积奖励

目录

- 机器学习基本概念
- 有监督学习：决策树和随机森林、kNN
- 无监督学习：聚类

决策树

根据树状结构进行分类

Q1: 这款游戏属于哪种类型?

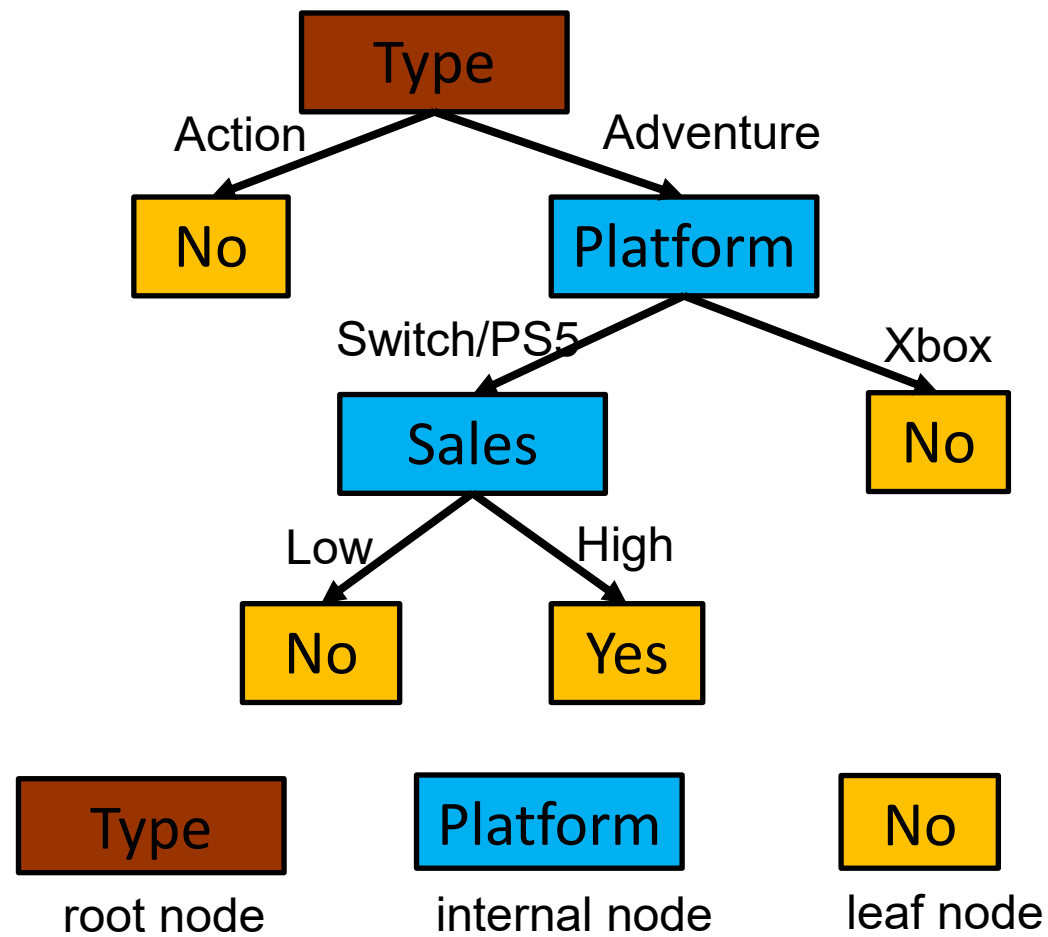
- 动作类: 不买
- 冒险类 - 转到Q2

Q2: 游戏的平台是什么?

- Xbox: 不买
- Switch/PS5 - 转到Q3

Q3: 销量是否很高?

- 不买
- 买



从根节点开始构造决策树

- 根节点
 - ✓ 整个训练集 $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$
 - ✓ 每个 x_i 由 d 维特征/属性描述
 - ✓ $A = \{a_1, a_2, \dots, a_d\}$ 属性名集合
- Example
 - ✓ 训练集包括10个样本
 - 如 $x_1 = [\text{Action}, \text{Xbox}, \text{High}]$, $y_1 = \text{No}$
 - ✓ $A = \{\text{Type}, \text{Platform}, \text{Sales}\}$

Type	Platform	Sales	Buy?
Action	Xbox	High	No
Action	Switch	Low	No
Action	PS5	High	No
Adventure	Switch	High	Yes
Adventure	Switch	High	Yes
Adventure	Switch	High	No
Adventure	Switch	Low	No
Adventure	PS5	High	Yes
Adventure	PS5	High	Yes
Adventure	PS5	High	No

如果未达到停止条件，则分裂根节点

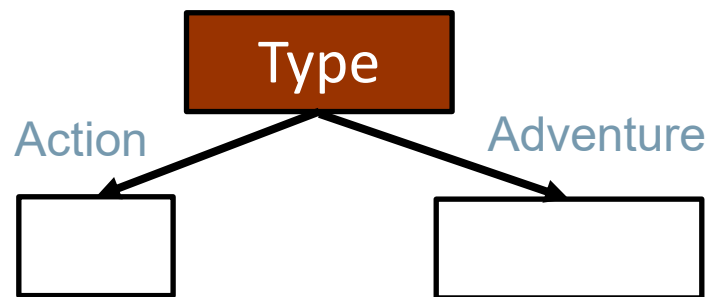
- 从 A 中选择最优的分裂特征 a_*
- 对于 a_* 的所有可能值，生成一个子节点，并创建与该子节点相关联的子集 $D_v \subset D$ 使得 D_v 中的所有样本有 $a_* = v$
- 对于所有子节点，修改它们的属性集合为 $A_v = A \setminus \{a_*\}$

根节点的停止条件

- 在 D 中的所有示例在 A 上具有相同的值 -> 将节点标记为 D 中的多数类
- 在 D 中的所有示例都属于同一类别 C -> 将节点标记为类别 C

分裂根节点

- 是否达到了停止条件？
- 假设 Type 是当前分裂的最优属性



TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

分裂非根节点

每个非根节点都与一个数据子集 D_v 和一个特征子集 A_v 关联

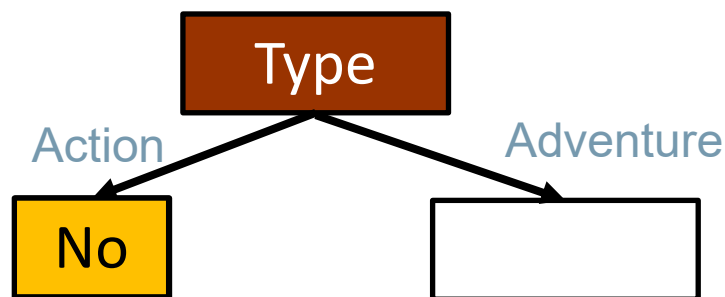
当 D_v 和 A_v 均非空：和根节点一样判断是否属于同一个类别或相同属性（停止条件），然后使用与分裂根节点相同的方法分裂非根节点

当 D_v 或 A_v 为空：停止分裂

- D_v 为空：将该非根节点标记为叶节点，并将其决策结果设置为其父节点中的多数类别
- A_v 为空：将该非根节点标记为叶节点，并将其决策结果设置为 D_v 中的多数类别

分裂非根节点

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No



TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

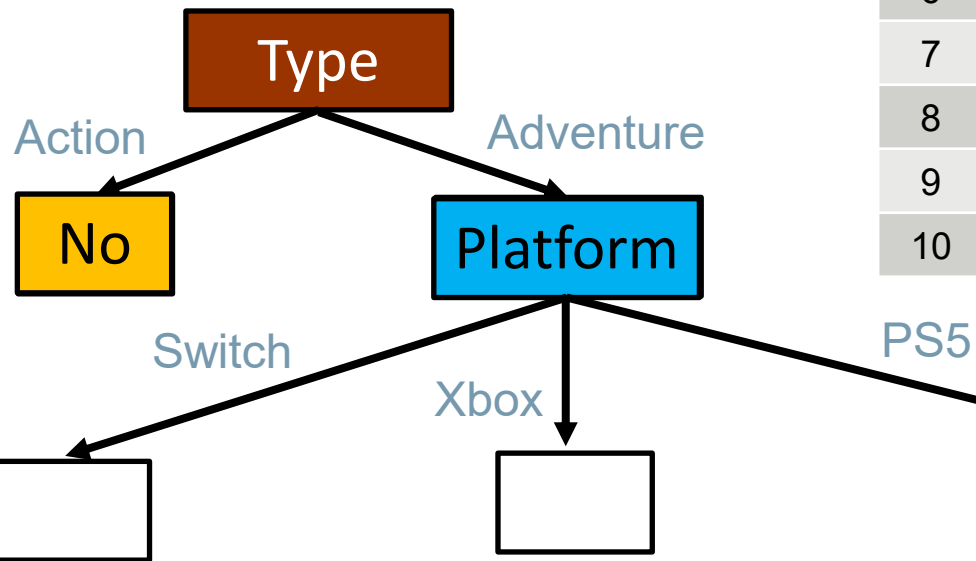
- 均非空： D_v 中样本都有同样的 A_v ? D_v 中样本都属于同样的 C ?

分裂非根节点



1 2 3 4 5 6 7 8 9 10

TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No



TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No

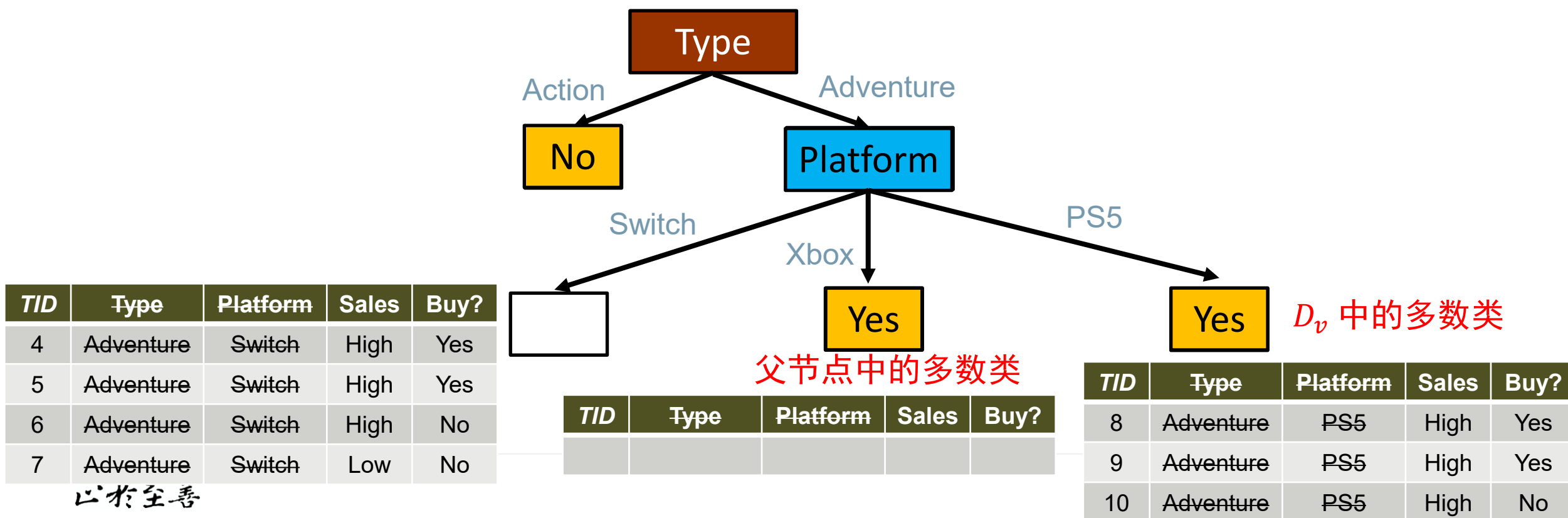
TID	Type	Platform	Sales	Buy?

TID	Type	Platform	Sales	Buy?
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

- 假设 Platform 是当前分裂的最优属性

分裂非根节点

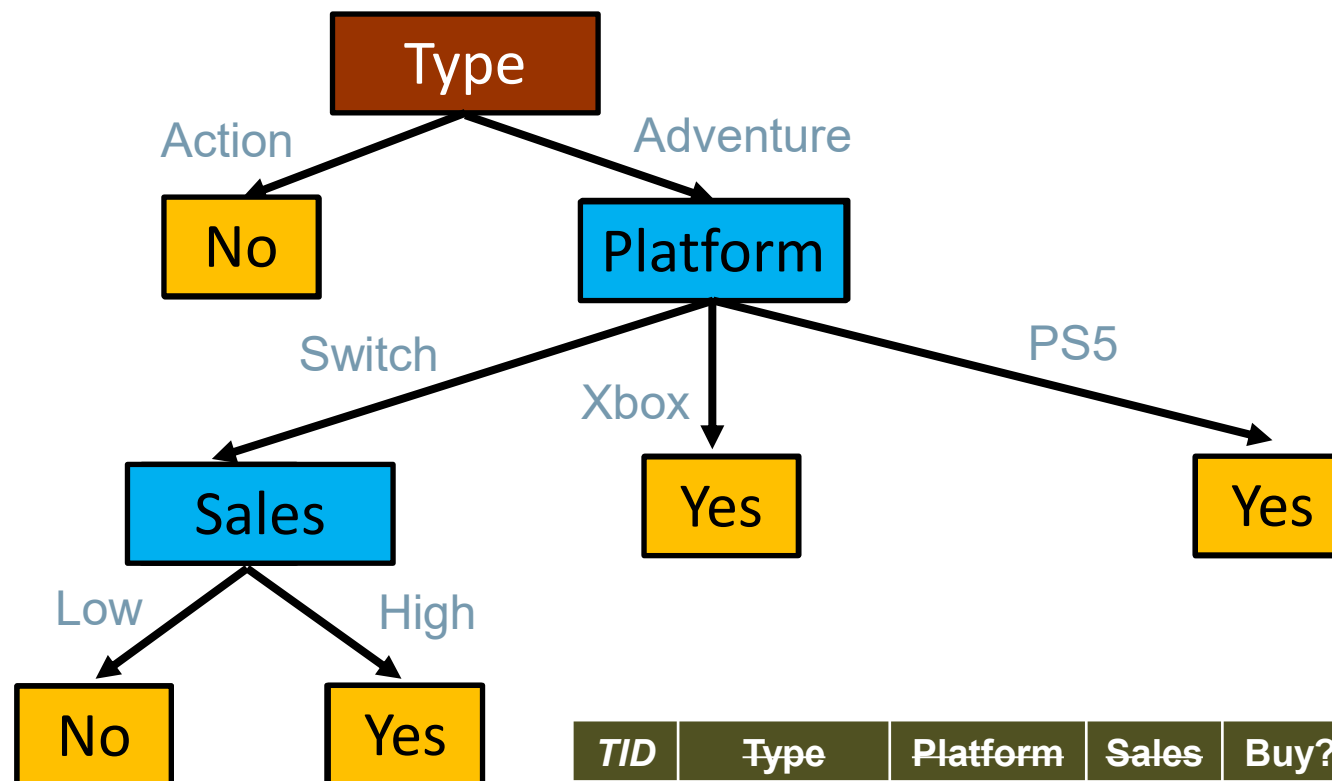
- 均非空: D_v 中样本都有同样的 A_v ? D_v 中样本都属于同样的 C ?
- D_v 是否为空?
- A_v 是否为空?



分裂非根节点

TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No

TID	Type	Platform	Sales	Buy?
7	Adventure	Switch	Low	No



TID	Type	Platform	Sales	Buy?
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No

构建决策树



输入：训练集 $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ ，属性集 $A = \{a_1, \dots, a_d\}$

TreeGenerate(D, A)

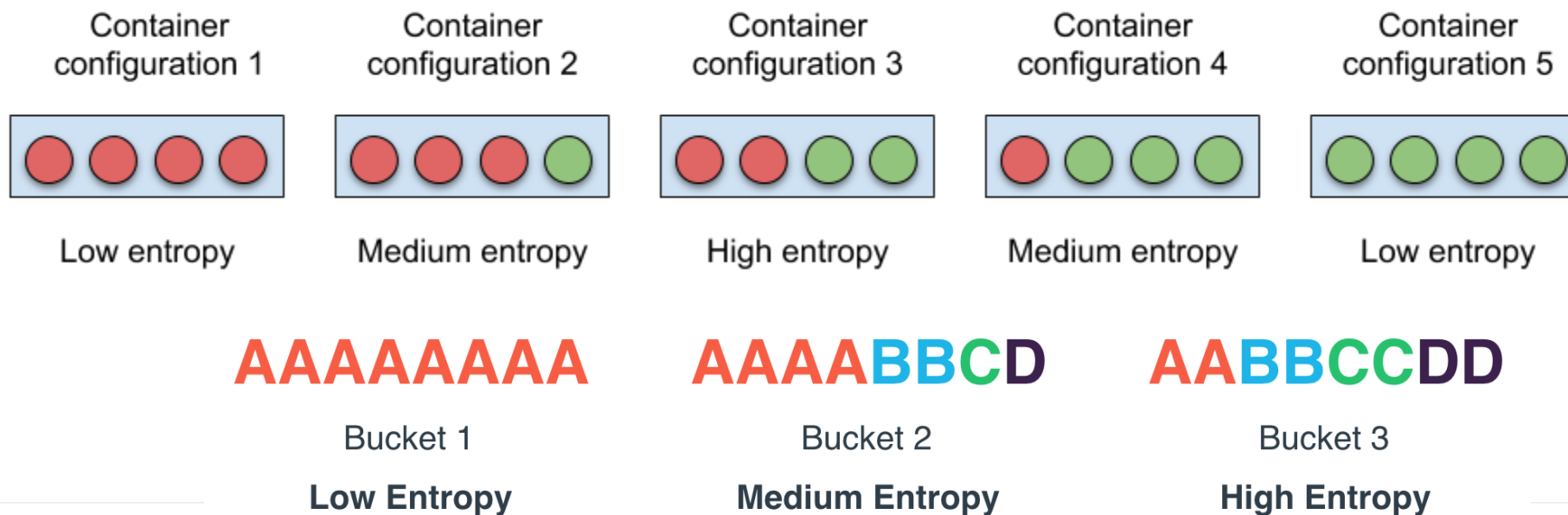
1. 如果满足停止准则：将当前节点设为叶节点并标记其类别，返回
2. 从 A 中选择最优的分裂特征 a_*
3. 对于 a_* 的所有可能值，生成子节点，并创建与该子节点相关联的子集 $D_v \subset D$ 使得 D_v 中的所有样本有 $a_* = v$
4. 对于所有子集 D_v ：调用 TreeGenerate($D_v, A \setminus \{a_*\}$)

停止准则：

1. 当前节点的所有样本属于同一类别 C ，将该节点标记为类别 C
2. 特征集 A 为空，或 D 中所有样本在 A 上的取值相同，将该节点标记为 D 中的多数类别
3. 数据集 D 为空，将该叶节点标记为其父节点数据中的多数类别

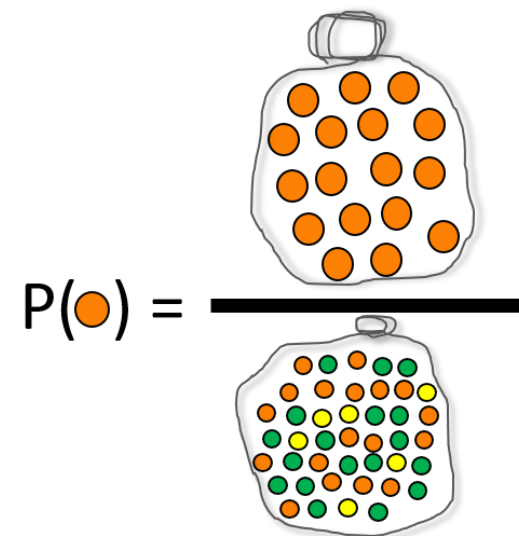
如何选择最优分裂属性

- 原则：随着拆分过程的进行，我们希望每个子节点内的样本有越来越倾向于属于同一类，即纯度越来越高
- 在统计学中，**熵**通常被视为“无序程度”的一种度量



- 假设有 m 类: $C_1, C_2, \dots, C_m$
- p_i : D 中属于类别 C_i 的比例
- 信息熵 (if $p = 0, p \log_2 p = 0$)

$$\text{Ent}(D) = - \sum_{i=1}^m p_i \log_2 p_i$$



- $\text{Ent}(D)$ 越小，纯度越高。最小值为0，最大值为 $\log_2 m$

- Yes: 4
- No: 6

$$\text{Ent}(D) = - \sum_{i=1}^m p_i \log_2 p_i$$

$$\text{Ent}(\{1 - 10\}) = -\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} = 0.9710$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

- 直观解释：通过特征 a 对数据集 D 进行分割后，熵值减少
- 假设特征 a 有 V 个可能取值 $\{v_1, v_2, \dots, v_V\}$
- 根据 a 进行节点分裂
- D_{v_j} ：所有 $a = v_j$ 的样本
- 选择**信息增益最大**的特征分裂节点：

$$\text{Gain}(D, a) = \text{Ent}(D) - \sum_{j=1}^V \frac{|D_{v_j}|}{|D|} \text{Ent}(D_{v_j})$$

先算出每个 D_{v_j} 的信息熵，考虑到不同分支节点包含的样本数不同，给分支赋予权重，样本越多的分支节点影响越大，然后算出信息增益

信息增益

Type:

- Act.

$$\text{Ent}(\{1 - 3\}) = -\frac{3}{3}\log_2 \frac{3}{3} - \frac{0}{3}\log_2 \frac{0}{3} = 0$$

- Adv.

$$\text{Ent}(\{4 - 10\}) = -\frac{4}{7}\log_2 \frac{4}{7} - \frac{3}{7}\log_2 \frac{3}{7} = 0.9852$$

- 信息增益

$$\text{Gain}(\{1 - 10\}, \text{Type}) = 0.9710 - \frac{3}{10} \times 0 - \frac{7}{10} \times 0.9852 = 0.2814$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

Platform:

- Xbox

$$\text{Ent}(\{1\}) = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$$

- Switch

$$\text{Ent}(\{2, 4 - 7\}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.9710$$

- PS5

$$\text{Ent}(\{3, 8 - 10\}) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1$$

- 信息增益

$$\text{Gain}(\{1 - 10\}, \text{Platform}) = 0.9710 - \frac{1}{10} \times 0 - \frac{5}{10} \times 0.9710 - \frac{4}{10} \times 1 = 0.0855$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

Sales:

- High

$$\text{Ent}(\{1, 3 - 6, 8 - 10\}) = -\frac{4}{8}\log_2 \frac{4}{8} - \frac{4}{8}\log_2 \frac{4}{8} = 1$$

- Low

$$\text{Ent}(\{2, 7\}) = -\frac{2}{2}\log_2 \frac{2}{2} - \frac{0}{2}\log_2 \frac{0}{2} = 0$$

- 信息增益

$$\text{Gain}(\{1 - 10\}, \text{Sales}) = 0.9710 - \frac{2}{10} \times 0 - \frac{8}{10} \times 1 = 0.1710$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

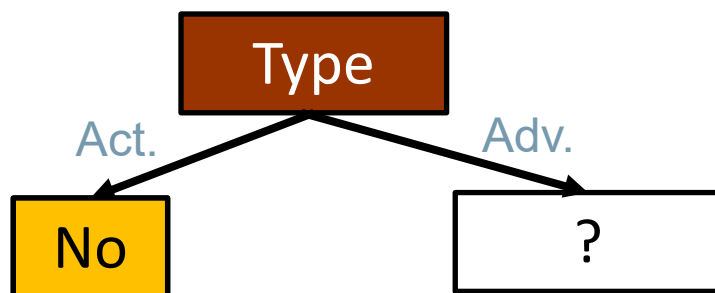
- 选择信息增益最大的特征分裂节点

$$\text{Gain}(\{1 - 10\}, \text{Type}) = 0.9710 - \frac{3}{10} \times 0 - \frac{7}{10} \times 0.9852 = 0.2814$$

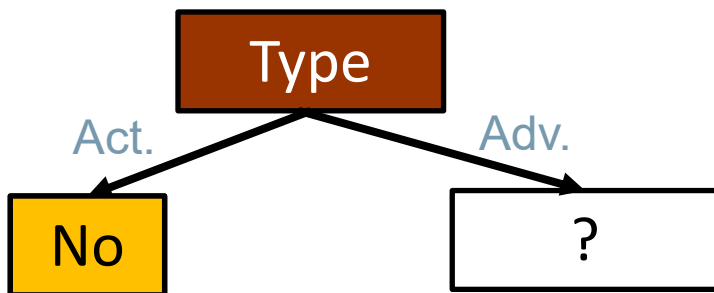
$$\text{Gain}(\{1 - 10\}, \text{Platform}) = 0.9710 - \frac{1}{10} \times 0 - \frac{5}{10} \times 0.9710 - \frac{4}{10} \times 1 = 0.0855$$

$$\text{Gain}(\{1 - 10\}, \text{Sales}) = 0.9710 - \frac{2}{10} \times 0 - \frac{8}{10} \times 1 = 0.1710$$

信息增益



TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No



TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

- 子节点的熵

$$\text{Ent}(\{4 - 10\}) = -\frac{4}{7}\log_2\frac{4}{7} - \frac{3}{7}\log_2\frac{3}{7} = 0.9852$$

信息增益

Platform:

- Switch

$$\text{Ent}(\{4 - 7\}) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1$$

- PS5

$$\text{Ent}(\{8 - 10\}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

- 信息增益

$$\text{Gain}(\{4 - 10\}, \text{Platform}) = 0.9852 - \frac{4}{7} \times 1 - \frac{3}{7} \times 0.9183 = 0.0202$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

信息增益

Sales:

- High

$$\text{Ent}(\{7\}) = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$$

- Low

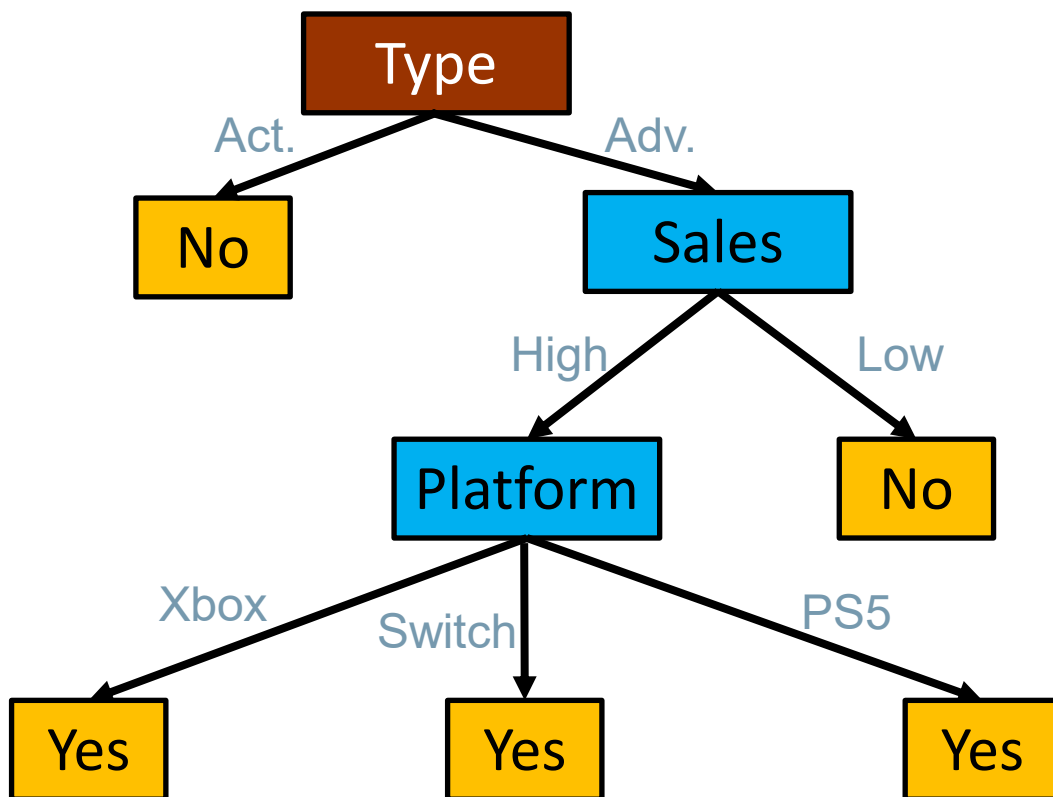
$$\text{Ent}(\{4-6, 8-10\}) = -\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6} = 0.9183$$

- 信息增益

$$\text{Gain}(\{4-10\}, \text{Sales}) = 0.9852 - \frac{1}{7} \times 0 - \frac{6}{7} \times 0.9183 = 0.1980$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

信息增益



TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

信息增益的偏好

TID:

$$\text{Ent}(\{1\}) = -\frac{1}{1}\log_2 \frac{1}{1} - \frac{0}{1}\log_2 \frac{0}{1} = 0$$

$$\text{Gain}(\{1 - 10\}, \text{TID}) = 0.9710 - 10 \times \frac{1}{10} \times 0 = 0.9710$$

$$\text{Gain}(\{1 - 10\}, \text{Type}) = 0.2814$$

$$\text{Gain}(\{1 - 10\}, \text{Platform}) = 0.0855$$

$$\text{Gain}(\{1 - 10\}, \text{Sales}) = 0.1710$$

信息增益对可取值数多的属性有偏好

固有值

- 假设特征 a 有 V 个可能取值 $\{v_1, v_2, \dots, v_V\}$
- 根据 a 进行节点分裂
- D_{v_j} : 所有 $a = v_j$ 的样本

- 固有值:

$$IV(a) = - \sum_{j=1}^V \frac{|D_{v_j}|}{|D|} \log_2 \frac{|D_{v_j}|}{|D|}$$

- 通常取值数多就大

固有值

- TID

$$IV(TID) = - \sum_1^{10} \frac{1}{10} \log_2 \frac{1}{10} = 3.3219$$

- Type

$$IV(\text{Type}) = - \frac{3}{10} \log_2 \frac{3}{10} - \frac{7}{10} \log_2 \frac{7}{10} = 0.8813$$

- Sales

$$IV(\text{Sales}) = - \frac{2}{10} \log_2 \frac{2}{10} - \frac{8}{10} \log_2 \frac{8}{10} = 0.7219$$

- Platform

$$IV(\text{Platform}) = - \frac{1}{10} \log_2 \frac{1}{10} - \frac{4}{10} \log_2 \frac{4}{10} - \frac{5}{10} \log_2 \frac{5}{10} = 1.3610$$

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

- 增益率：

$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(D, a)}$$

- 选择**增益率最大**的特征分裂节点
- 其目的是降低选择那些容易产生更多分支的特征的可能性
- 增益率对可取值数少的属性有偏好
- 因此C4.5并不是直接选择增益率大的属性来划分，而是用了启发式：先从候选划分属性中寻找信息增益高出平均水平的属性，再从中选择增益率高的

$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(D, a)}$$

$$\begin{aligned} \text{Gain}(\{1 - 10\}, \text{TID}) &= 0.9710 - 10 \times \frac{1}{10} \times 0 = 0.9710 & \text{IV}(\text{TID}) &= 3.3219 \\ \text{Gain}(\{1 - 10\}, \text{Type}) &= 0.2814 & \text{IV}(\text{Type}) &= 0.8813 \\ \text{Gain}(\{1 - 10\}, \text{Platform}) &= 0.0855 & \text{IV}(\text{Platform}) &= 1.3610 \\ \text{Gain}(\{1 - 10\}, \text{Sales}) &= 0.1710 & \text{IV}(\text{Sales}) &= 0.7219 \end{aligned}$$

	<i>TID</i>	Type	Platform	Sales
Gain_ratio	0.2923	0.3193	0.0628	0.2369

- 在决策树中，如果分支过多，可能会被训练数据的“特殊特征”所误导 -> 过拟合
- 剪枝：在决策树中剪掉一些分支，以降低过拟合的风险
 - ✓ 预剪枝：提早停止继续构造树。如果分裂节点会导致较差的泛化性就不要分裂
 - ✓ 后剪枝：从“完全生长”的树中删除分支。如果删除会导致更好的泛化性就删除
 - ✓ 后剪枝通常比预剪枝保留更多的分支，欠拟合风险很小，泛化性能优，但时间开销大



预剪枝

- 在树的构建过程中，选择Type作为下一个分裂属性

训练集

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	No
6	Adventure	Switch	High	No
7	Adventure	Xbox	Low	Yes
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

验证集

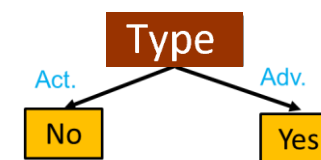
TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Adventure	Switch	Low	Yes
3	Action	PS5	High	No

不分裂

- 标注为多数类别 No
- 在验证集上准确率为 0.67

用Type分裂

- 在验证集上准确率为 1



预剪枝

- 在树的构建过程中，选择Platform作为下一个分裂属性

训练集

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	No
6	Adventure	Switch	High	No
7	Adventure	Xbox	Low	Yes
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

验证集

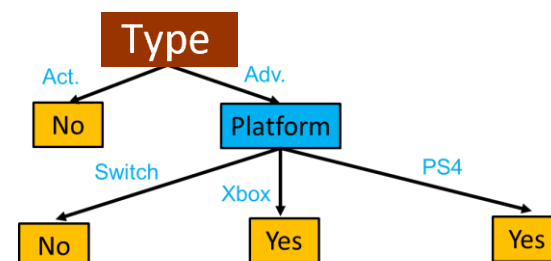
TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Adventure	Switch	Low	Yes
3	Action	PS5	High	No

不分裂

- 在验证集上准确率为 1

用Platform分裂

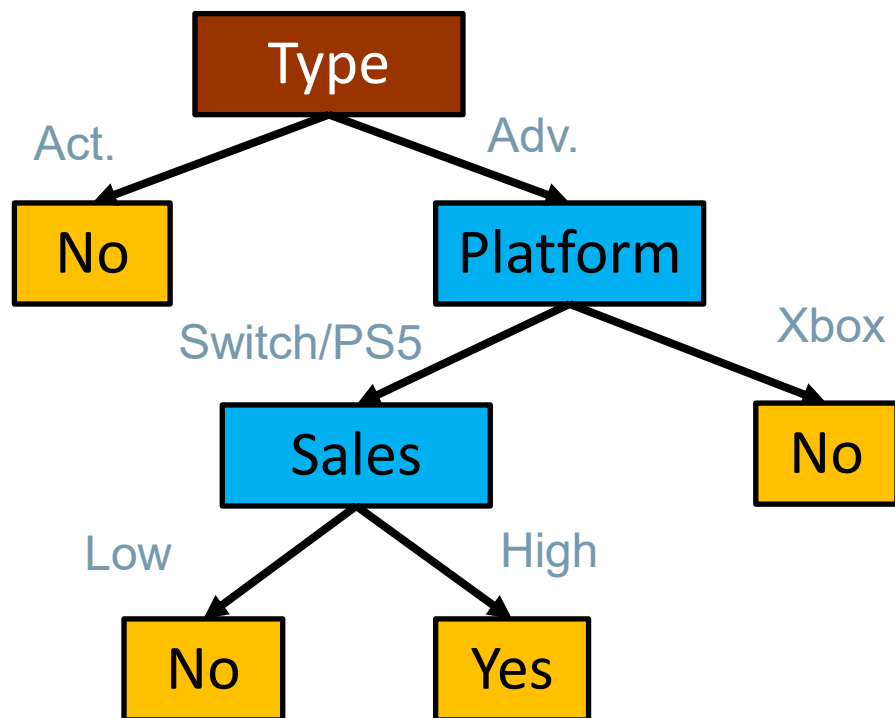
- 在验证集上准确率为 0.67



剪枝!

后剪枝

- 树构建完成，验证集准确率0.67



Validation data

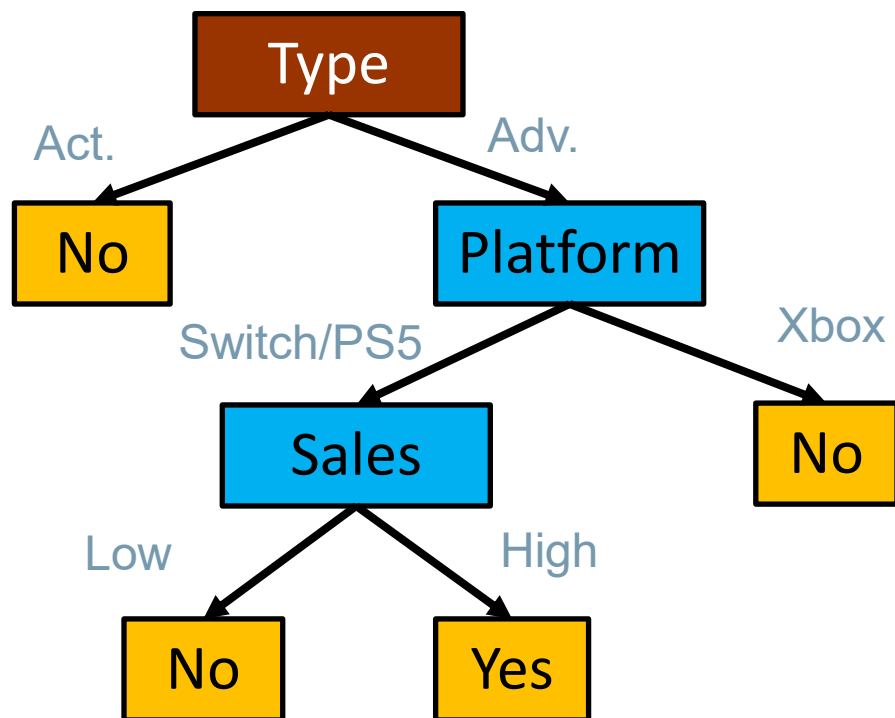
TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Adventure	Switch	Low	Yes
3	Action	PS5	High	No

Training data

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Action	Switch	Low	No
3	Action	PS5	High	No
4	Adventure	Switch	High	Yes
5	Adventure	Switch	High	Yes
6	Adventure	Switch	High	No
7	Adventure	Switch	Low	No
8	Adventure	PS5	High	Yes
9	Adventure	PS5	High	Yes
10	Adventure	PS5	High	No

后剪枝

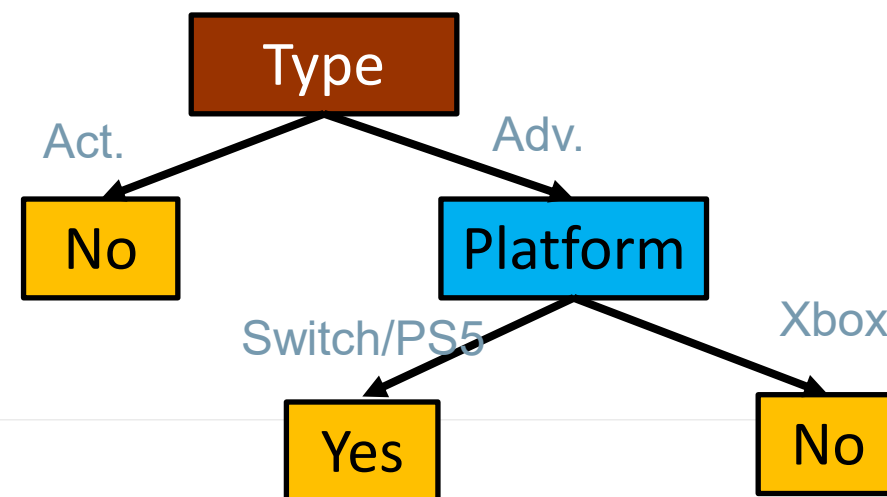
- 树构建完成，验证集准确率0.67



验证集

TID	Type	Platform	Sales	Buy?
1	Action	Xbox	High	No
2	Adventure	Switch	Low	Yes
3	Action	PS5	High	No

剪枝Sales，验证集准确率为1



集成学习

- 每个学习器被称为基学习器
- 集成学习：组合多个基分类器的输出，如多数表决



	testing sample 1	testing sample 2	testing sample 3		testing sample 1	testing sample 2	testing sample 3		testing sample 1	testing sample 2	testing sample 3
h_1	✓	✓	×	h_1	✓	✓	×	h_1	✓	×	×
h_2	×	✓	✓	h_2	✓	✓	×	h_2	×	✓	×
h_3	✓	×	✓	h_3	✓	✓	×	h_3	×	×	✓
Ensemble	✓	✓	✓	Ensemble	✓	✓	×	Ensemble	×	×	×

(a) Improved performance.

(b) Unchanged performance.

(c) Degenerated performance.

集成学习何时才能实现更优的预测效果？

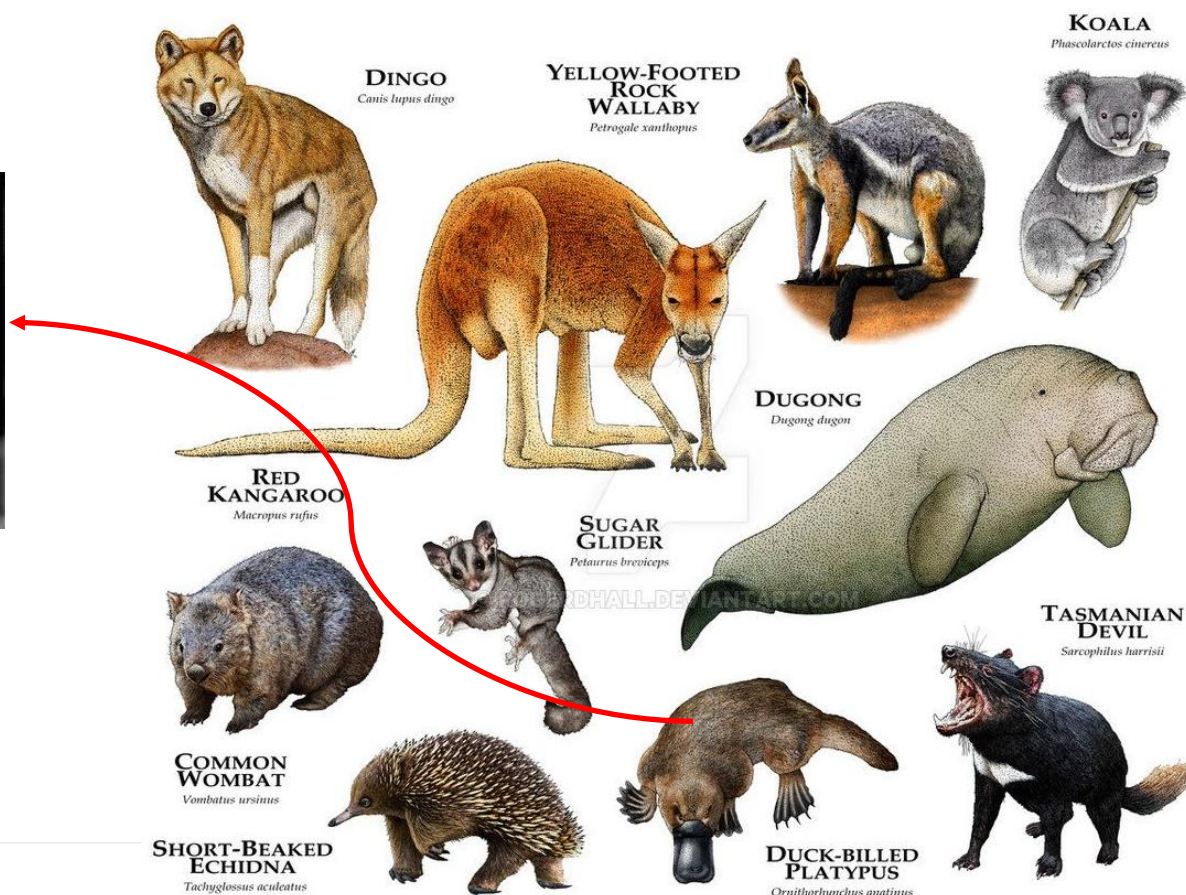
- 每个单独的分类器都表现良好
- 各个分类器之间存在差异性

随机森林

- 构建多棵决策树
- 为了引入随机性：在构建决策树的过程中，每棵树用一部分随机采样的样本，在节点分裂阶段只从随机抽取的 k 个特征中选择最优分裂特征
- 用多数投票的方式结合多棵树的输出

k近邻分类器 kNN

- 基本思想：从历史记录中找出与新记录最为相似的那些内容



k近邻分类器 kNN

对每个测试样本：

- 计算其与所有训练样本的距离
- 确定前 k 个最近邻样本
- 利用最近邻样本的标记来确定测试样本的标记，例如通过多数表决
- k近邻分类器（简称kNN）是一种惰性学习方法，因为在训练阶段它只是对训练样本进行了记忆，而没有构建任何模型
- 出于同样的原因，它也被称为基于示例的学习

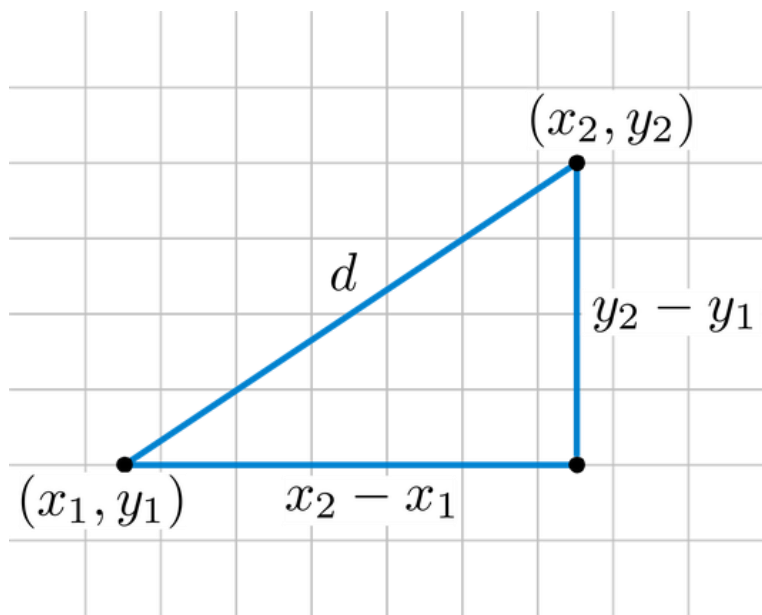
k近邻分类器 kNN



需要三个输入：

- 训练集
- 距离度量方法
- k （超参数）

- 通过距离来衡量的两个对象的相似性，越相似它们之间的距离就越小
- 最常用的是欧几里得距离



$$d = \sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2}$$

Minkowski 距离

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}|^2 \right)^{1/2}$$

Euclidean Distance

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}|^p \right)^{1/p}$$

Minkowski Distance

Minkowski 距离

- $p = 1$: Manhattan distance (L_1 distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}| \right)$$

- $p = 2$: Euclidean distance (L_2 distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}|^2 \right)^{1/2}$$

- $p = \infty$: Chebyshev distance (L_∞ distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \max_{u=\{1,2,\dots,d\}} |x_{iu} - x_{ju}|$$

k近邻分类器 kNN

Customer	Age	Income (K)	Games played	Buy?	Euclidean Distance from David
John	35	35	3	Yes	$\sqrt{(35 - 37)^2 + (35 - 50)^2 + (3 - 2)^2} = 15.16$
Rachel	22	50	2	No	$\sqrt{(22 - 37)^2 + (50 - 50)^2 + (2 - 2)^2} = 15$
Ruth	63	200	1	No	$\sqrt{(63 - 37)^2 + (200 - 50)^2 + (1 - 2)^2} = 152.23$
Tom	59	170	1	No	$\sqrt{(59 - 37)^2 + (170 - 50)^2 + (1 - 2)^2} = 122$
Neil	25	40	4	Yes	$\sqrt{(25 - 37)^2 + (40 - 50)^2 + (4 - 2)^2} = 15.74$
David	37	50	2	?	

- $k = 1$: 最近邻是 Rachel, 所以预测值是 No
- $k = 3$: 最近邻是 Rachel, John, and Neil, 所以预测值是 Yes
- $k = 5$: 最近邻是 Rachel, John, Neil, Tom and Ruth., 所以预测值是 No

- 属性值可能需要进行缩放处理，以避免距离测量结果被某一属性所主导
- 例如：
 - ✓ 一个人的年龄可能在 0 岁到 100 岁之间变化
 - ✓ 一个人的收入可能在 1000 元到 100 万之间波动
 - ✓ 一个人参加的比赛数量可能在 0 次到 1000 次之间变化

在训练阶段所应用的数据标准化方法也应在测试阶段使用！

选择 k

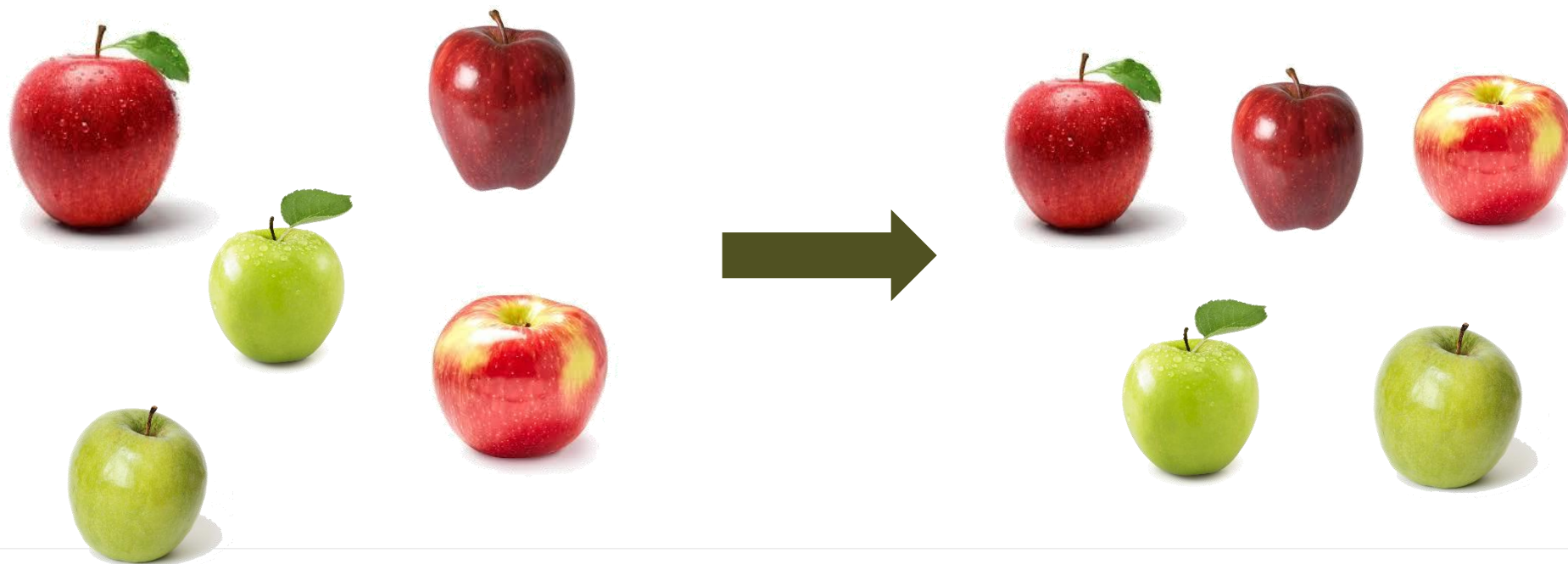
- 如果 k 值过小，那么一旦出现噪声点，预测结果就很容易出现偏差（对噪声敏感）
- 如果 k 值过大，那么邻域中可能会包含来自其他类别的过多点
- 通过验证集选择
- 选择奇数

- 将非数值特征转换为数值形式，如需可对训练数据进行标准化处理
- 指定超参数 k 和距离度量方法，并存储所有训练数据
- 对于测试样本，如有必要则进行标准化处理
- 计算其与所有训练实例的距离
- 获取训练数据中 k 个最近邻样本的所有标记，然后通过多数投票（或其他投票方法）进行预测

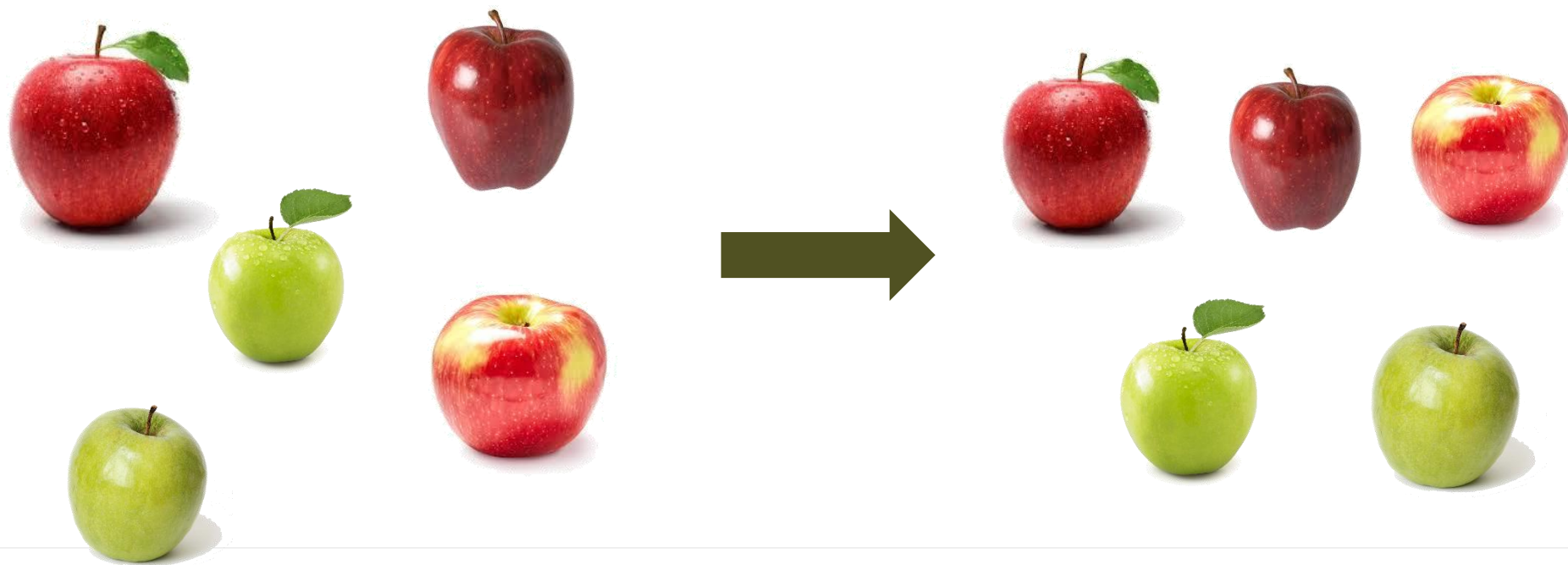
目录

- 机器学习基本概念
- 有监督学习：决策树和随机森林、kNN
- 无监督学习：聚类

- 给定一组对象，每个对象都有其属性的**描述**，根据这些描述将它们划分成不同的**组**，使得同一组内的对象具有**相似性**，而不同组的对象则具有**差异性**



- 给定一组对象，每个对象都有其属性的**描述**，根据这些描述将它们划分成不同的组，使得同一组内的对象具有相似性，而不同组的对象则具有差异性



特征

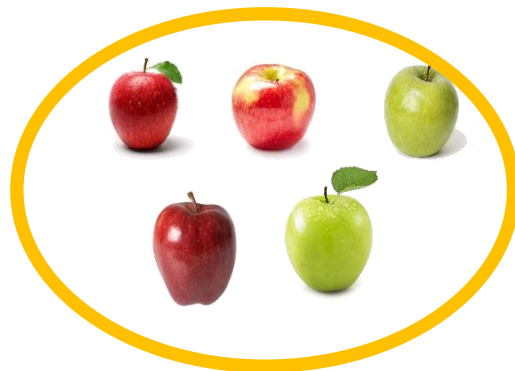
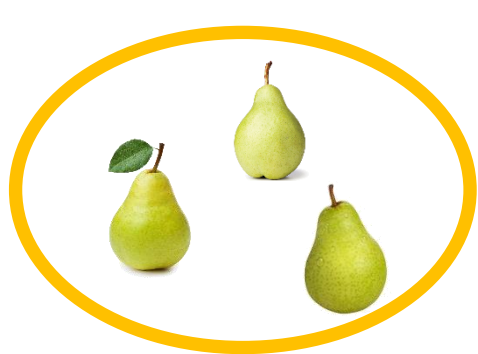
- 特征用于描述对象



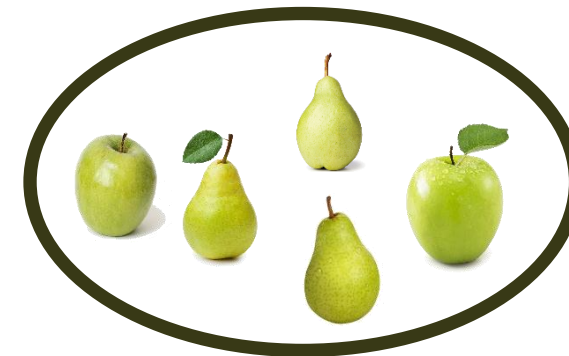
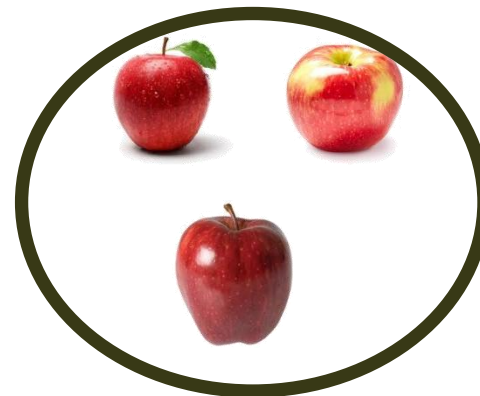
Colour	Taste
Green	Sour

- 利用领域知识从原始数据中提取特征的过程可能会对后续性能产生影响





根据品种聚类

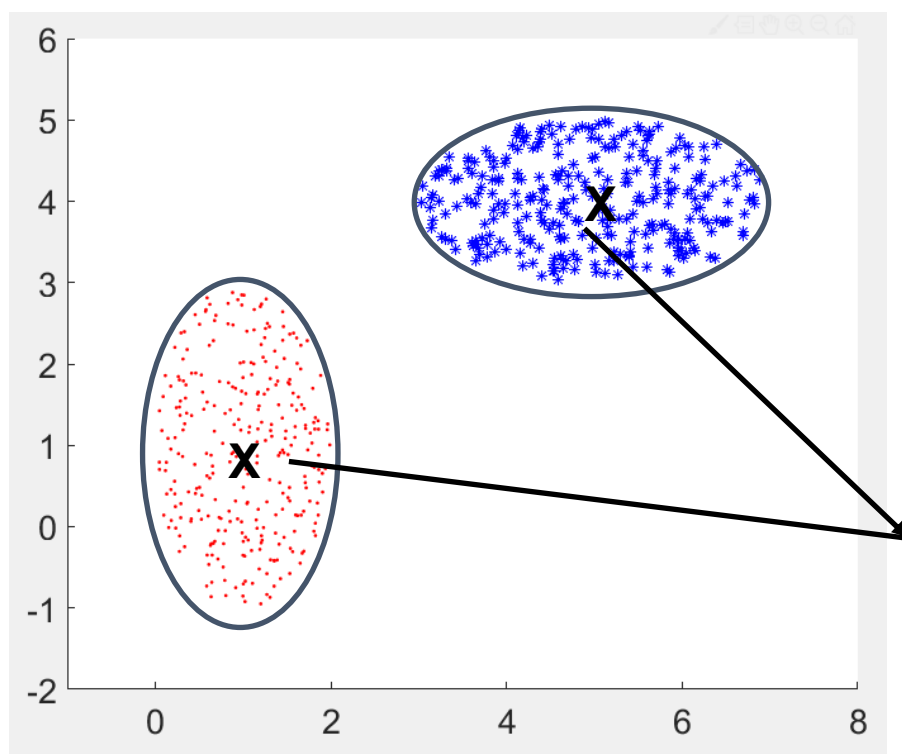


根据颜色聚类

我们对对象的描述方式至关重要，它可能会影响聚类的结果！

簇

- 一个组被称为簇
- 聚类中心：该簇内所有点的平均值



聚类中心

- 通过距离来衡量的两个对象的相似性，越相似，它们之间的距离就越小
- 在同一组内：点与点之间的距离较短
- 在不同组间：点与点之间的距离较长

Minkowski 距离

- $p = 1$: Manhattan distance (L_1 distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}| \right)$$

- $p = 2$: Euclidean distance (L_2 distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^d |x_{iu} - x_{ju}|^2 \right)^{1/2}$$

- $p = \infty$: Chebyshev distance (L_∞ distance)

$$\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \max_{u=\{1,2,\dots,d\}} |x_{iu} - x_{ju}|$$

距离度量的比较

- 如果大多数属性都很相近，但有少数属性却差异很大
- 哪种距离度量方法更有可能将这两者判定为“相似”？
- 哪种距离度量方法更有可能将这两者判定为“不相似”？

距离度量的比较

- $P1$ 和谁更近?

			Manhattan	Euclidean	Chebyshev
P1	[1, 2, 3, 4, 5]	$dist(P1, P2)$	$\sqrt{5}$	$\sqrt{5}$	$\sqrt{5}$
P2	[1, 2, 3, 4, $5-\sqrt{5}$]	$dist(P1, P3)$	5	$\sqrt{5}$	1
P3	[2, 3, 4, 5, 6]				

- Manhattan: P2
- Euclidean: 一样
- Chebyshev: P3

距离度量的比较

- 如果大多数属性都很相近，但有少数属性却差异很大
- 哪种距离度量方法更有可能将这两者判定为“相似”？

Manhattan

- 哪种距离度量方法更有可能将这两者判定为“不相似”？

Chebyshev

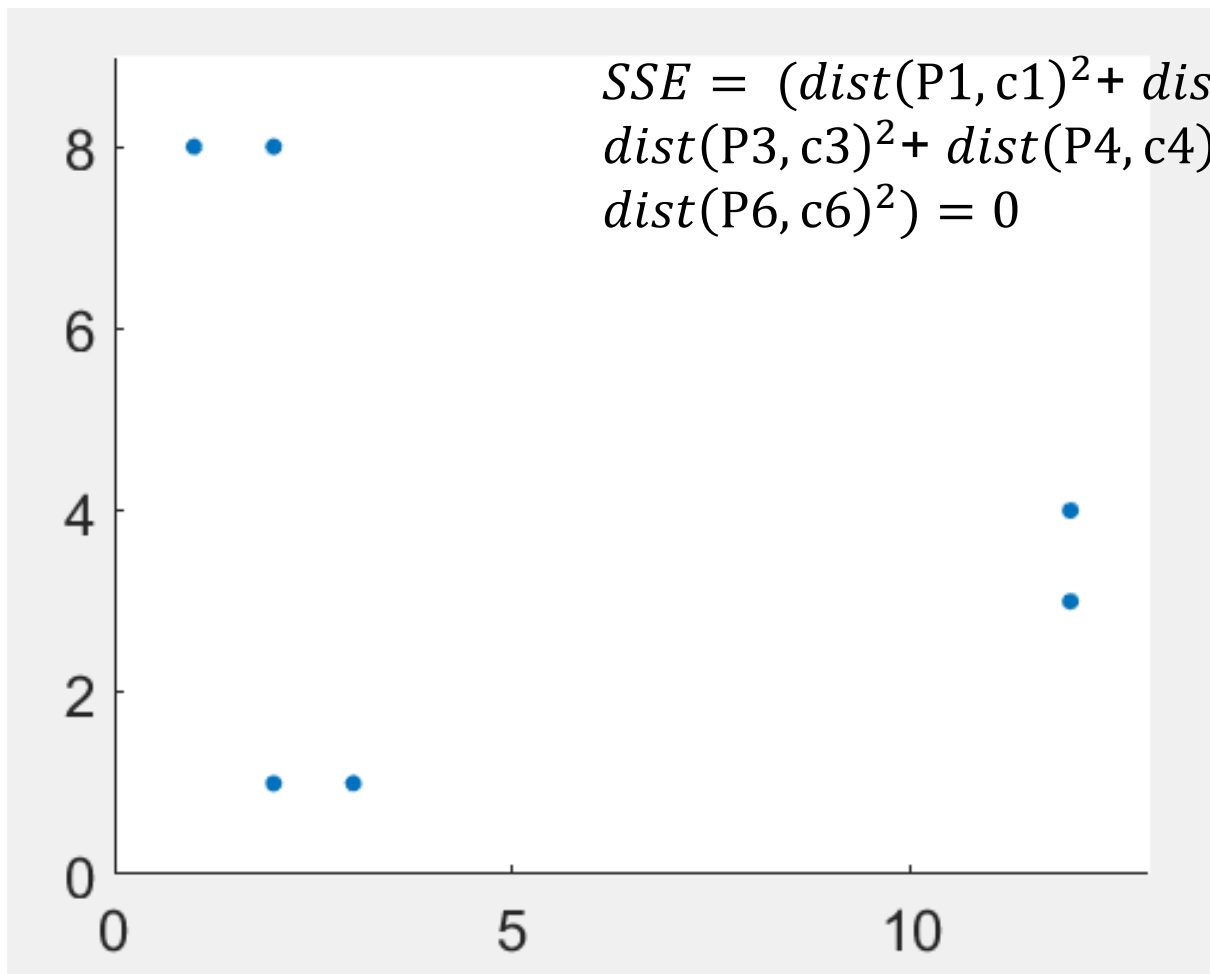
平方误差总和 SSE

- 如果数据被聚类成 k 个簇，每个簇分别表示为 C_i
- 每个簇的中心为 c_i

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} dist(x, c_i)^2$$

- 我们希望使SSE值减小，但由不同的k值计算出的SSE值是无法进行比较的

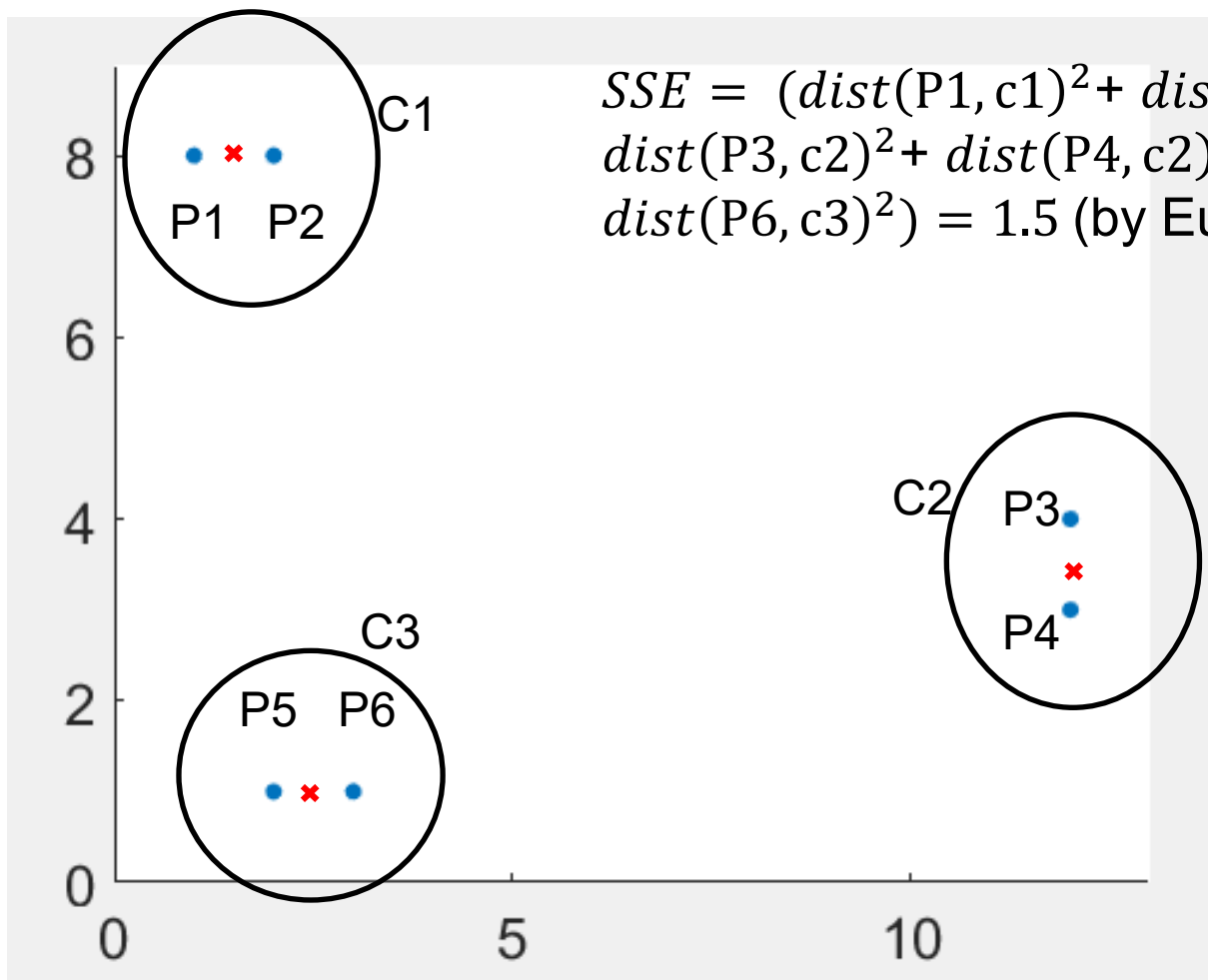
平方误差总和 SSE



$$SSE = (dist(P1, c1)^2 + dist(P2, c2)^2 + dist(P3, c3)^2 + dist(P4, c4)^2 + dist(P5, c5)^2 + dist(P6, c6)^2) = 0$$

	x_1	x_2
P1	1	8
P2	2	8
P3	12	4
P4	12	3
P5	2	1
P6	3	1

平方误差总和 SSE



	x_1	x_2
P1	1	8
P2	2	8
P3	12	4
P4	12	3
P5	2	1
P6	3	1

	x_1	x_2
c1	1.5	8
c2	12	3.5
c3	2.5	1

k-Means

- 训练集 $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- 输出: k个聚类
- 目标: 最小化SSE

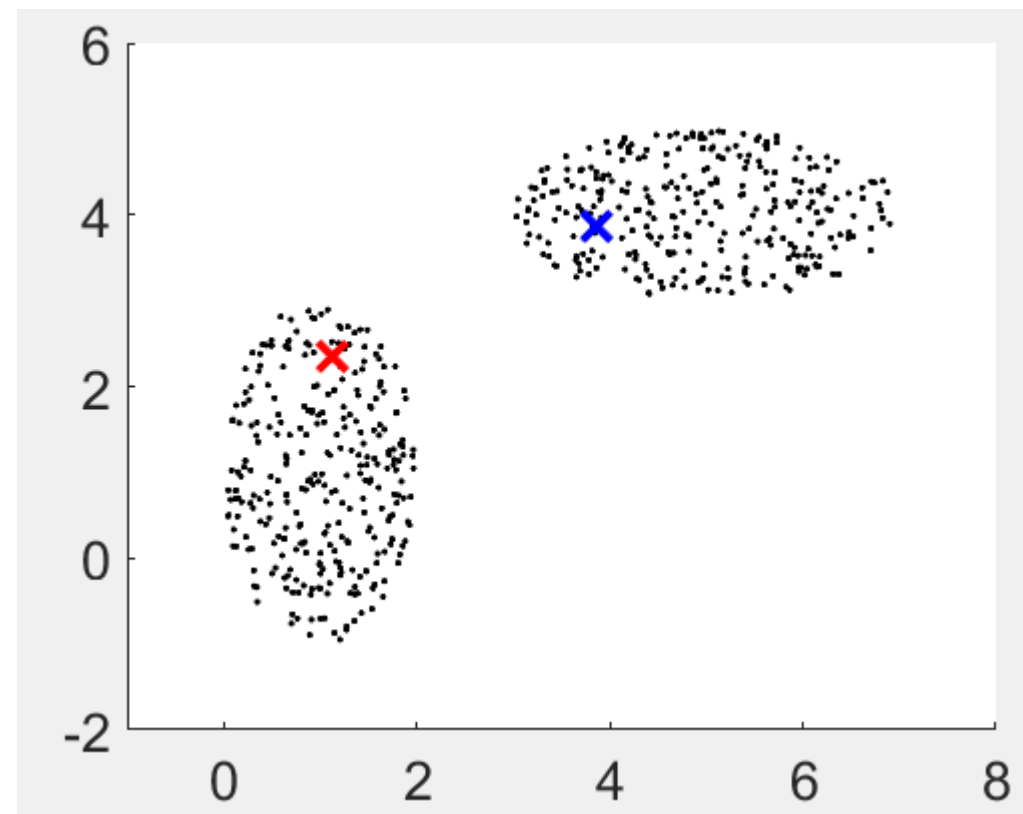
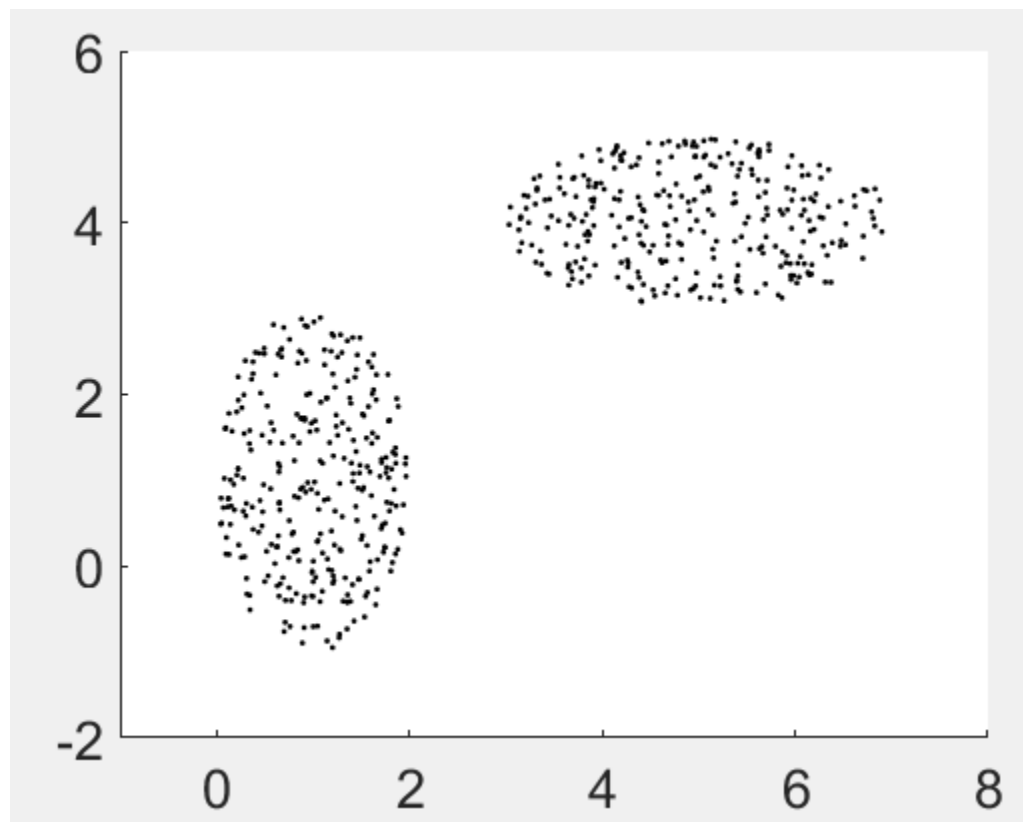
$$\operatorname{argmin}_{C_1, C_2, \dots, C_k} \sum_{j=1}^k \sum_{x_i \in C_j} \operatorname{dist}(x_i, c_j)^2$$

这是在科学和工业应用中使用最为广泛的聚类算法

- 初始化：随机选取 k 个点作为初始聚类中心
- 重复以下步骤，直至簇的分配不再改变
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 k 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值

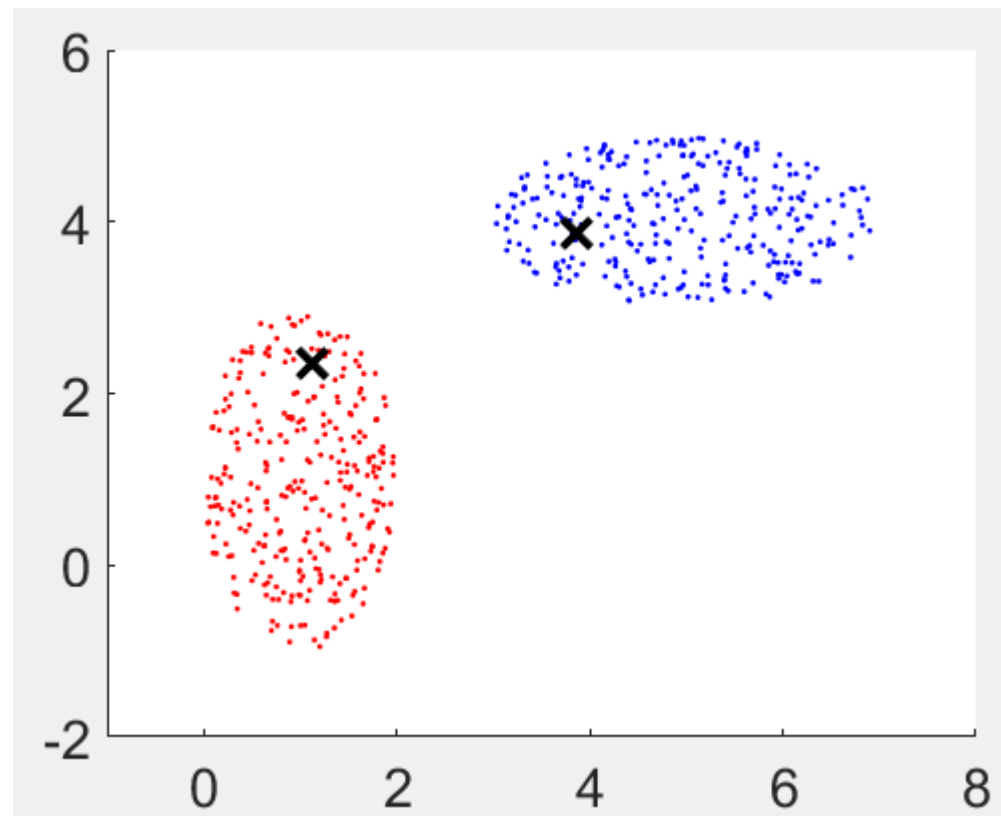
k-Means

初始化：随机选取2个点作为初始聚类中心



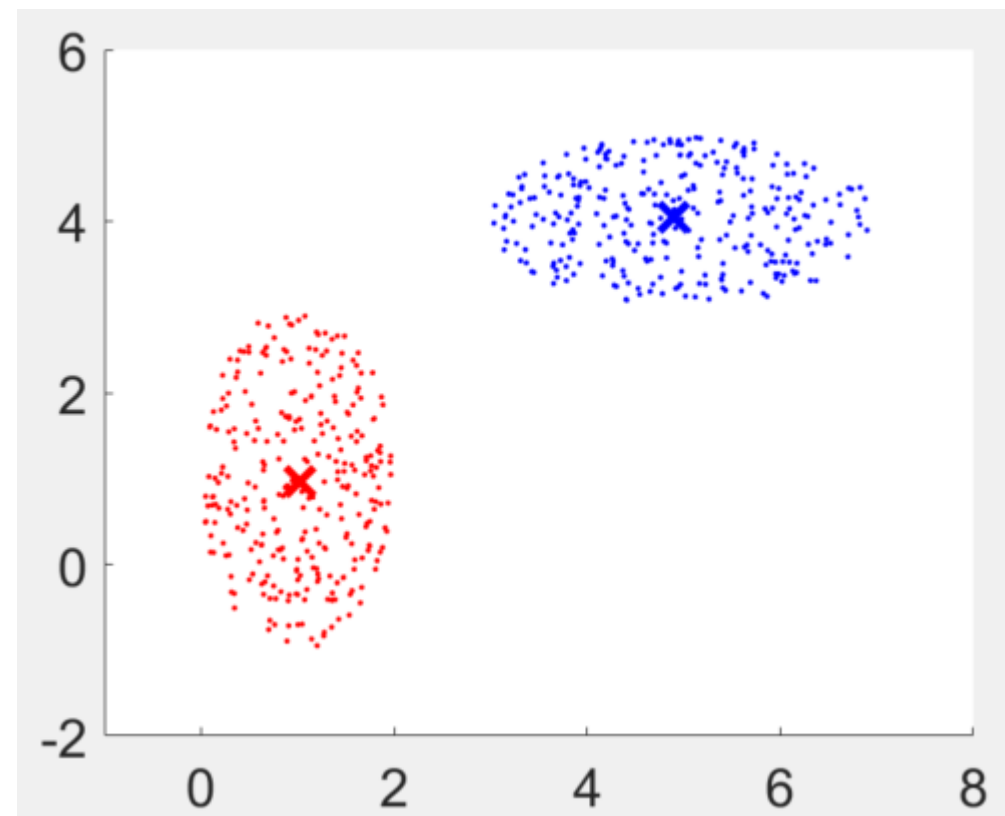
k-Means

- 重复以下步骤，直至簇的分配不再改变（第一轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



k-Means

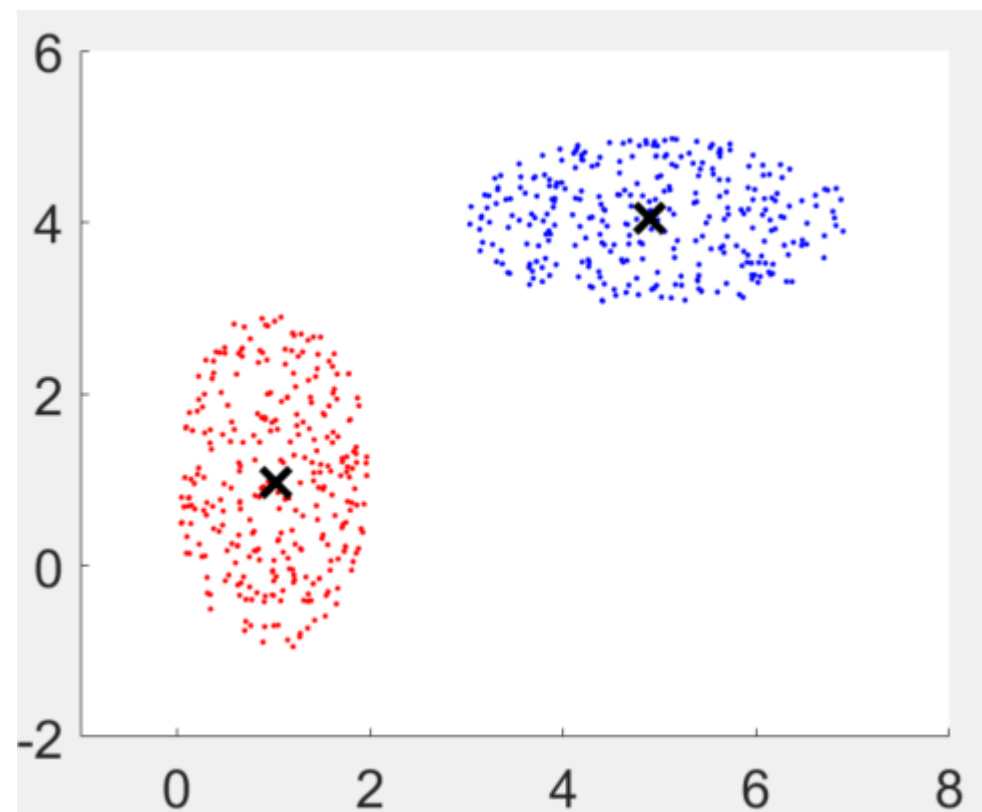
- 重复以下步骤，直至簇的分配不再改变（第一轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



k-Means

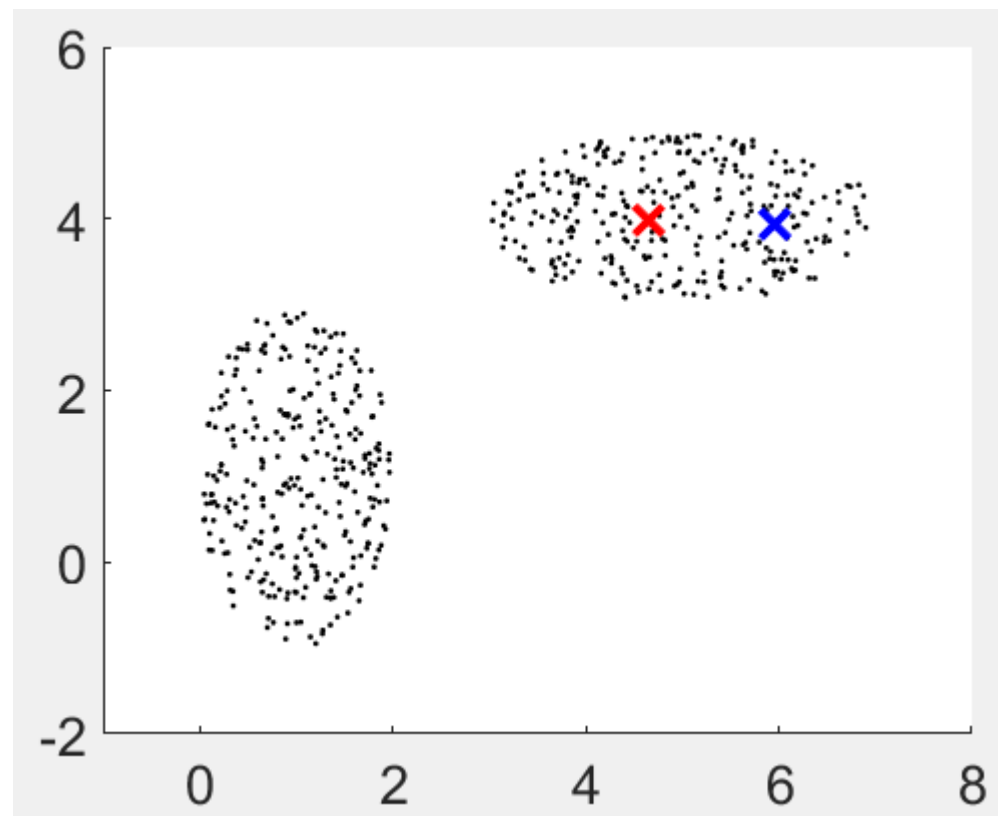
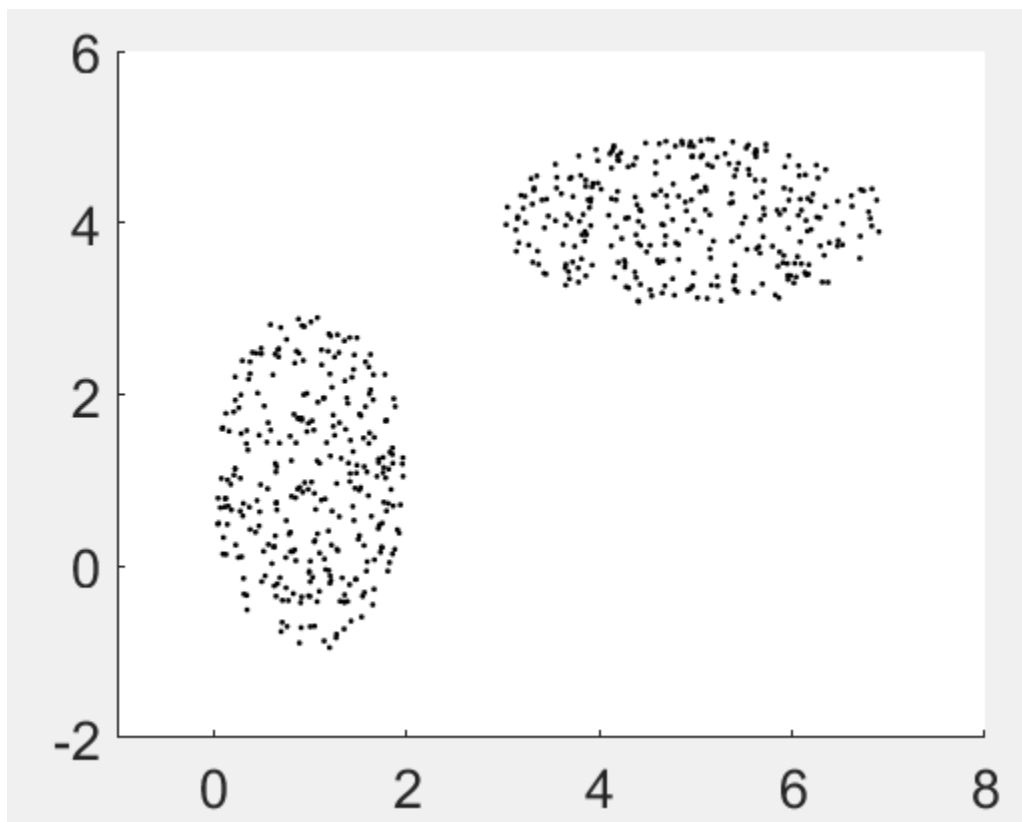
- 重复以下步骤，直至簇的分配不再改变（第二轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值

簇的分配不再改变，迭代结束



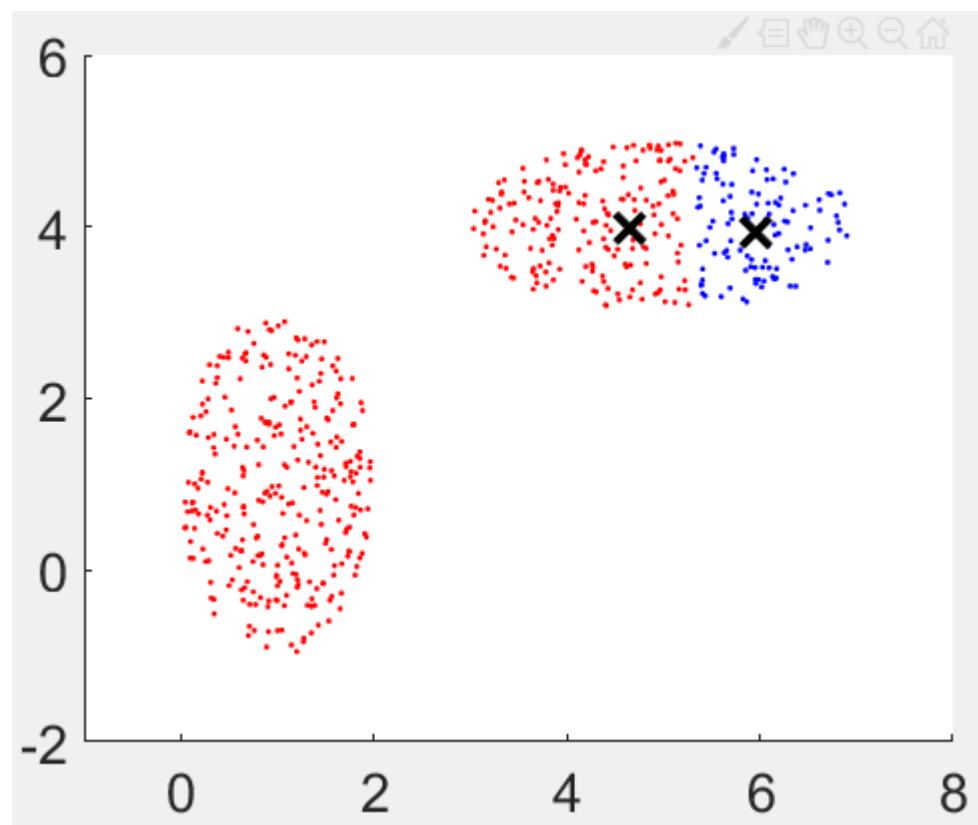
k-Means

初始化：随机选取2个点作为初始聚类中心



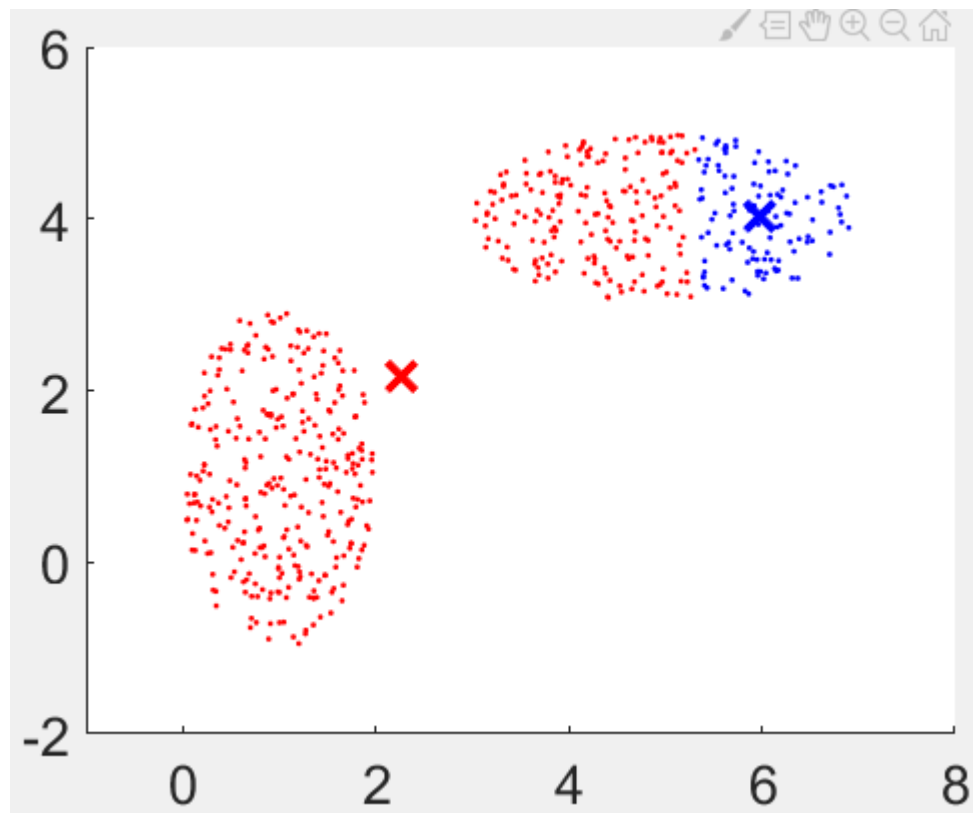
k-Means

- 重复以下步骤，直至簇的分配不再改变（第一轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



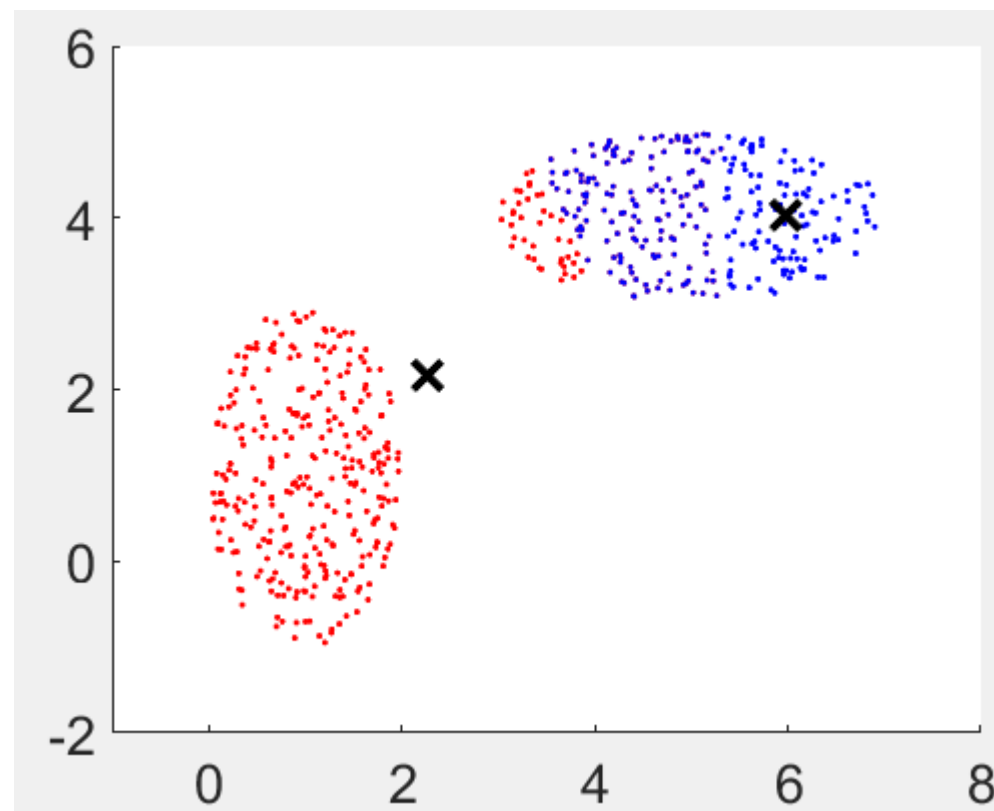
k-Means

- 重复以下步骤，直至簇的分配不再改变（第一轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



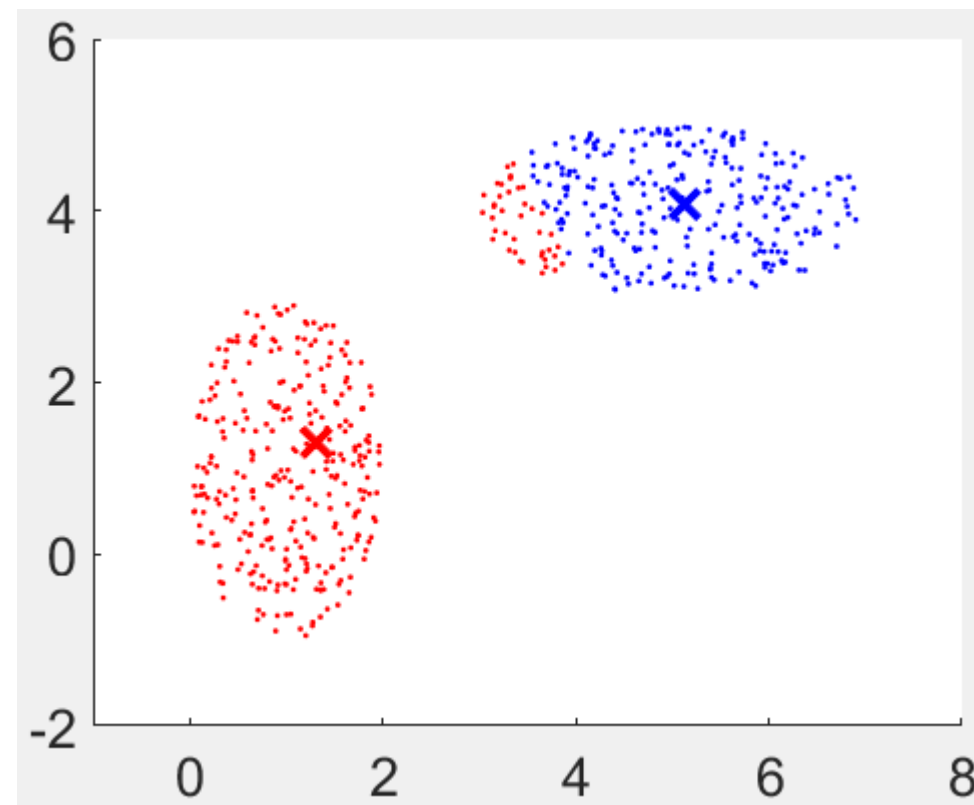
k-Means

- 重复以下步骤，直至簇的分配不再改变（第二轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



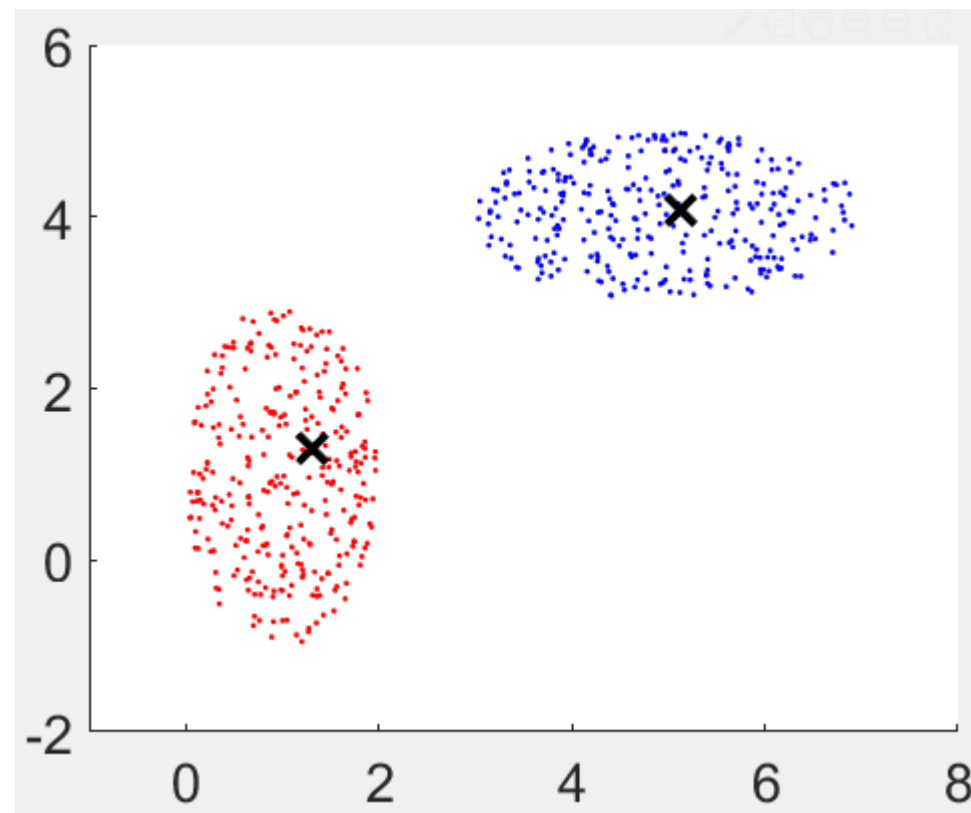
k-Means

- 重复以下步骤，直至簇的分配不再改变（第二轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



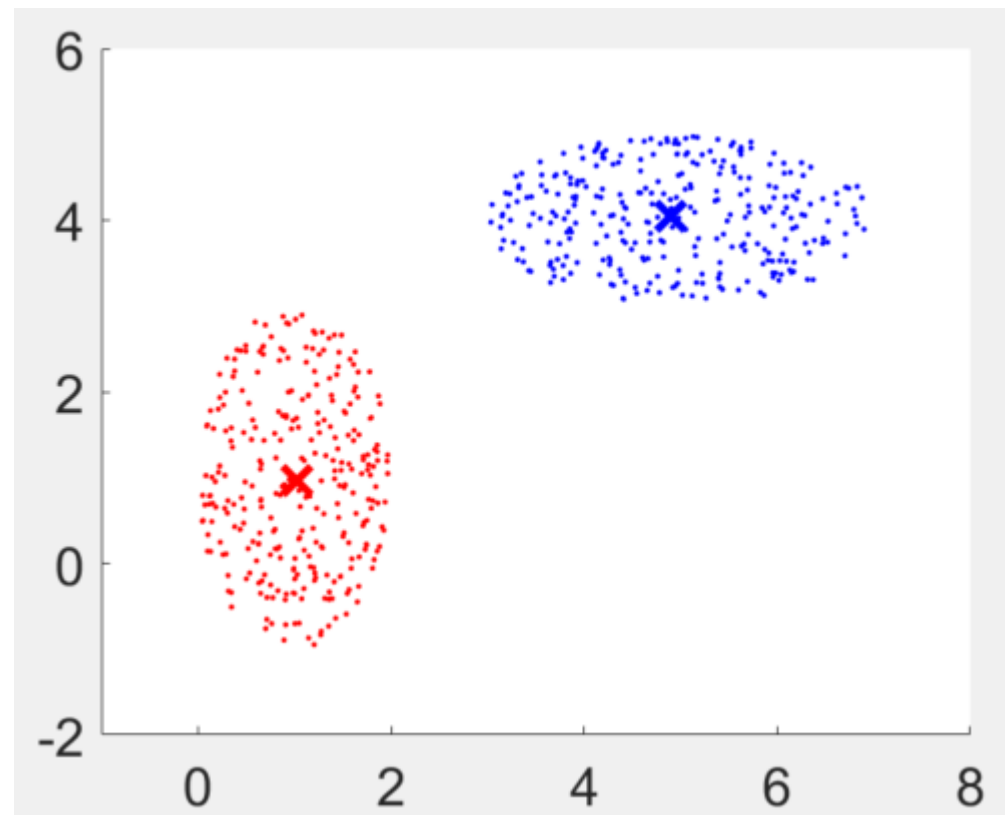
k-Means

- 重复以下步骤，直至簇的分配不再改变（第三轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



k-Means

- 重复以下步骤，直至簇的分配不再改变（第三轮）
 1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇
 2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值



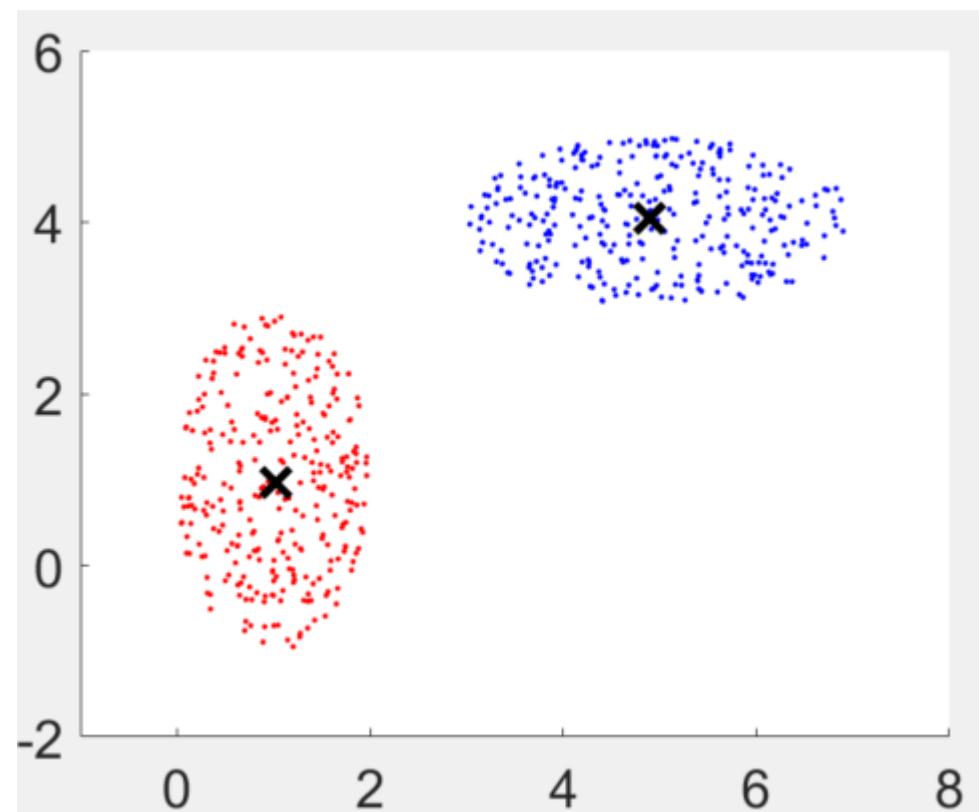
k-Means

- 重复以下步骤，直至簇的分配不再改变（第四轮）

1. 更新所属对象：通过将点分配到最近的聚类中心，形成 2 个簇

2. 更新聚类中心：将每个簇的质心设置为该簇内所有点的平均值

簇的分配不再改变，迭代结束



k-Means

- 初始化可能会影响最终结果 -> k-Means++
- 尝试不同的k, k的取值通常在2到 $\sqrt{n}$ 之间
- 算法在数据形成球形簇的情况下表现良好, 但对于没有这种特征的数据则效果不佳

